


UNIVERSITÀ
DEGLI STUDI
FIRENZE

# *Elaborato Calcolo Numerico 2025-2026*

Autore: Lorenzo Yang

Matricola: 7136074

E-mail: lorenzo.yang@edu.unifi.it

Giugno 2026

# Indice

Esercizio 1 . . . 3
Esercizio 2 . . . 4
Esercizio 3 . . . 6
Esercizio 4 . . . 8
Esercizio 5 . . . 10
Esercizio 6 . . . 13
Esercizio 7 . . . 13
Esercizio 8 . . . 15
Esercizio 9 . . . 17
Esercizio 10 . . . 20
Esercizio 11 . . . 22
Esercizio 12 . . . 24
Esercizio 13 . . . 27
Esercizio 14 . . . 29
Esercizio 15 . . . 32
Esercizio 16 . . . 35
Esercizio 17 . . . 36
Esercizio 18 . . . 38
Esercizio 19 . . . 40
Esercizio 20 . . . 41
Esercizio 21 . . . 43
Esercizio 22 . . . 45
Esercizio 23 . . . 47
Esercizio 24 . . . 48
Esercizio 25 . . . 54
Esercizio 26 . . . 55
Esercizio 27 . . . 57
Esercizio 28 . . . 59
Esercizio 29 . . . 61
Esercizio 30 . . . 63

# Esercizio 1

> Combinare linearmente $y(t+h), y(t), hy'(t), hy'(t-h), hy'(t-2h)$, con coefficienti non nulli e indipendenti da $h$, in modo da ottenere un residuo che sia $O(h^4)$. Dettagliare il procedimento.

## Svolgimento

Combiniamo i termini assegnati in modo che, negli sviluppi di Taylor, si annullino tutti i contributi di ordine inferiore a $h^4$.

### Sviluppo di Taylor dei termini

Espandiamo funzioni e derivate fino al terzo ordine; i resti confluiscono in $O(h^4)$:

- $y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \frac{h^3}{6}y'''(t) + O(h^4)$
- $y(t) = y(t)$
- $hy'(t) = hy'(t)$
- $hy'(t-h) = h\Big(y'(t) - hy''(t) + \frac{h^2}{2}y'''(t) + O(h^3)\Big) = hy'(t) - h^2y''(t) + \frac{h^3}{2}y'''(t) + O(h^4)$
- $hy'(t-2h) = h\Big(y'(t) - 2hy''(t) + \frac{(2h)^2}{2}y'''(t) + O(h^3)\Big) = hy'(t) - 2h^2y''(t) + 2h^3y'''(t) + O(h^4)$

### Definizione della combinazione lineare

Determiniamo i coefficienti $a, b, c, d, e$ tali che:

$$ay(t+h) + by(t) + chy'(t) + dhy'(t-h) + ehy'(t-2h) = O(h^4)$$

Sostituendo gli sviluppi e raccogliendo rispetto alle derivate di $y(t)$, imponiamo l'annullamento dei coefficienti dei termini di ordine inferiore a $h^4$:

- Termini in $y(t)$: $a + b = 0$
- Termini in $hy'(t)$: $a + c + d + e = 0$
- Termini in $h^2y''(t)$: $\frac{a}{2} - d - 2e = 0$
- Termini in $h^3y'''(t)$: $\frac{a}{6} + \frac{d}{2} + 2e = 0$

### Risoluzione del sistema

Il sistema è omogeneo, con 4 equazioni e 5 incognite. Fissiamo quindi un parametro libero; per evitare frazioni poniamo $a = 12$:

Dalla prima equazione:

$$12 + b = 0 \Rightarrow b = -12$$

Sostituendo $a = 12$ nella terza e quarta equazione, otteniamo un sottosistema in $d$ ed $e$:

$$\frac{12}{2} - d - 2e = 0 \Rightarrow d + 2e = 6$$

$$\frac{12}{6} + \frac{d}{2} + 2e = 0 \Rightarrow \frac{d}{2} + 2e = -2$$

Sottraendo la seconda equazione dalla prima:

$$(d + 2e) - \left(\frac{d}{2} + 2e\right) = 6 - (-2) \Rightarrow \frac{d}{2} = 8 \Rightarrow d = 16$$

Sostituendo $d = 16$ in $d + 2e = 6$:

$$16 + 2e = 6 \Rightarrow 2e = -10 \Rightarrow e = -5$$

Infine, dalla seconda equazione ricaviamo $c$:

$$12 + c + 16 - 5 = 0 \Rightarrow c + 23 = 0 \Rightarrow c = -23$$

#### Risultato finale

I coefficienti richiesti sono $a = 12$, $b = -12$, $c = -23$, $d = 16$, $e = -5$. La combinazione lineare finale è:

$$12y(t + h) - 12y(t) - 23hy'(t) + 16hy'(t - h) - 5hy'(t - 2h) = O(h^4)$$

## Esercizio 2

> Scrivere una function Matlab che, sapendo solamente che l'aritmetica finita implementata utilizza la base binaria con arrotondamento, calcoli: la precisione di macchina; il più grande numero di macchina; il più piccolo numero di macchina normalizzato; il più piccolo numero di macchina denormalizzato. Argomentare la sua stesura, che deve prevedere solo l'utilizzo di operazioni algebriche elementari (validare i risultati con le function `realmax` e `realmin` di Matlab).

### Svolgimento

#### Argomentazione teorica

- **Precisione di macchina ($u$)**: in base binaria con arrotondamento, la precisione è lo *unit roundoff*. Si calcola prima la distanza `eps1` tra 1 e il successivo numero di macchina, poi si pone $u = \frac{\text{eps1}}{2}$.
- **Più grande numero di macchina**: la massima mantissa rappresentabile in base 2 è $2 - \text{eps1}$. Moltiplicandola per 2 fino all'overflow si ottiene il massimo numero finito; il controllo usa il fatto che il reciproco dell'infinito macchina è zero.

- **Più piccolo numero normalizzato**: si parte da 1 e si dimezza finché il candidato conserva la precisione relativa dei normalizzati. Il test usa `eps1`, perché `eps1 / 2` è già sotto la distanza tra 1 e il numero macchina successivo.
- **Più piccolo numero denormalizzato**: dal minimo normalizzato si continua a dividere per 2.0 fino a ottenere zero. L'ultimo valore strettamente positivo è il minimo denormalizzato.

### Codice Matlab

```
function [u_mac, max_mac, min_norm, min_denorm] = calcola_costanti_macchina()
% Function [u_mac, max_mac, min_norm, min_denorm] = calcola_costanti_macchina()
%
% Calcola le costanti fondamentali della macchina assumendo che l'aritmetica
% finita implementata utilizzi la base binaria con arrotondamento.
% Il calcolo utilizza esclusivamente operazioni algebriche elementari.
% La function provvede inoltre a stampare a video la validazione dei
% risultati confrontandoli con le costanti native di Matlab.
%
% Input:
%   Nessuno
%
% Output:
%   u_mac      - precisione di macchina
%   max_mac    - piu' grande numero di macchina
%   min_norm   - piu' piccolo numero di macchina normalizzato
%   min_denorm - piu' piccolo numero di macchina denormalizzato
%
% Rel. 05-03-2026.

% 1. calcolo della distanza tra 1 e il numero di macchina successivo
eps1 = 1.0;
while (1.0 + eps1 / 2.0) > 1.0
    eps1 = eps1 / 2.0;
end
% con arrotondamento la precisione di macchina (unit roundoff) e' eps1/2
u_mac = eps1 / 2.0;

% 2. calcolo del piu' grande numero di macchina
max_mac = 2.0 - eps1;
while (1.0 / (max_mac * 2.0)) > 0.0
    max_mac = max_mac * 2.0;
end

% 3. calcolo del piu' piccolo numero di macchina normalizzato
min_norm = 1.0;
while ((min_norm / 2.0) * (1.0 + eps1)) > (min_norm / 2.0)
```

```
    min_norm = min_norm / 2.0;
end

% 4. Calcolo del piu' piccolo numero di macchina denormalizzato
min_denorm = min_norm;
while (min_denorm / 2.0) > 0.0
    min_denorm = min_denorm / 2.0;
end

% stampa e validazione dei risultati
fprintf('VALIDAZIONE COSTANTI DI MACCHINA\n');
fprintf('Precisione(u): calcolato=%e | Atteso(eps/2)=%e\n', u_mac, eps / 2);
fprintf('Realmax: calcolato=%e | Matlab(realmax)=%e\n', max_mac, realmax);
fprintf('Realmin Norm: calcolato=%e | Matlab(realmin)=%e\n', min_norm, realmin);
% il valore esatto corrisponde a realmin * eps
fprintf('Min Denorm: calcolato=%e | Atteso=%e\n', min_denorm, realmin * eps);

end
```

## Esercizio 3

Scrivere una function Matlab che implementi il metodo di Newton per gli zeri di una funzione. Prevedere valori di default per la tolleranza di arresto e per il numero massimo di iterazioni.

### Svolgimento

La funzione implementa il metodo di Newton, con parametro opzionale per la molteplicità della radice e criterio di arresto misto assoluto/relativo.

```
function [x, it] = metodoNewton(f, f1, x0, tol, maxit, molt)
% Function [x, it] = metodoNewton(f, f1, x0, tol, maxit, molt)
%
% Metodo di Newton (modificato) per gli zeri di funzione.
%
% Input:
%   f, f1 - function che implementano la funzione e la sua derivata;
%   x0    - punto iniziale;
%   tol   - tolleranza di arresto (default 1e-12);
%   maxit - numero massimo di iterazioni (default 1000);
%   molt  - molteplicita' della radice (default 1).
%
% Output:
```

```
%   x     - approssimazione della radice;
%   it    - numero di iterazioni effettuate.
%
% Rel. 05-03-2026.

% gestione degli argomenti di input e valori di default
if nargin < 3
    error('Numero di argomenti insufficiente.');
end
if nargin < 6
    molt = 1;
end
if nargin < 5
    maxit = 1000;
end
if nargin < 4
    tol = 1e-12;
end

% controllo validita' input
if molt < 1 || molt ~= fix(molt)
    error('Molteplicita'' non corretta: deve essere un intero positivo.');
end
if maxit < 1
    error('Numero massimo di iterateraZioni non corretto.');
end
if tol <= 0
    error('Tolleranza non corretta: deve essere > 0.');
end

% inizializzazione
x = x0;
flag = 1;
it = 0;

% ciclo iterativo di Newton
for i = 1:maxit
    it = i;
    x0 = x; % salvo il valore precedente

    % valutazione funzione e derivata
    fx = feval(f, x);
    f1x = feval(f1, x);

    % controllo se ho già trovato la radice esatta
    if fx == 0
        flag = 0;
```

```
        break;
    end

    % evito divisione per zero
    if f1x == 0
        error('Arresto: derivata nulla nel punto x = %g', x);
    end

    % aggiornamento passo di Newton modificato
    x = x0 - molt * (fx / f1x);

    % criterio di arresto (errore misto assoluto/relativo)
    if abs(x - x0) <= tol * (1 + abs(x))
        flag = 0;
        break;
    end
end

% avviso se non si è raggiunta la convergenza
if flag
    warning('Accuratezza non raggiunta entro il numero massimo di iterazioni.');
end

end
```

## Esercizio 4

Scrivere una function Matlab che implementi il metodo di bisezione. Prevedere valori di default per la tolleranza di arresto.

### Svolgimento

```
function [x, itnum] = metodoBisezione(f, a, b, tol)
% Function [x, itnum] = metodoBisezione(f, a, b, tol)
%
% Trova la radice di f in [a,b] usando il metodo di bisezione.
%
% Input:
%   f   - funzione per cui trovare la radice
%   a   - estremo sinistro dell'intervallo
%   b   - estremo destro dell'intervallo
%   tol - tolleranza per il criterio di arresto (default 1e-3)
```

```
%
% Output:
%   x     - stima della radice
%   itnum - numero di iterazioni effettuate
%
% Rel. 07-03-2026.

% gestione input e valori di default
if nargin < 3
    error('Numero di argomenti insufficiente.');
end

if nargin < 4
    tol = 1e-3;
end

if a >= b
    error('Intervallo [a, b] non valido.');
end

% valutazione iniziale e controllo dei segni
fa = f(a);
fb = f(b);

if fa == 0
    x = a; itnum = 0; return;
elseif fb == 0
    x = b; itnum = 0; return;
elseif fa * fb > 0
    error('Condizione f(a)*f(b) < 0 non soddisfatta.');
end

% calcolo del numero di iterazioni necessarie a priori
itmax = ceil(log2(b - a) - log2(tol));
itnum = 0;

% ciclo di bisezione
for k = 1:itmax
    itnum = itnum + 1;

    % calcolo del punto medio
    x = a + (b - a) / 2;
    fx = f(x);

    if fx == 0
        return;
    end
```

```
    % aggiornamento dell'intervallo
    if fa * fx < 0
        b = x;
    else
        a = x;
        fa = fx;
    end
end

end
```

## Esercizio 5

Scrivere una function Matlab che implementi un metodo ibrido: inizialmente il metodo di bisezione e, non appena l'ampiezza dell'intervallo di confidenza diventi minore dell'ampiezza prefissata $\delta$ (e se non si sia usciti in precedenza) passi ad utilizzare il metodo di Newton, a partire dall'ultima approssimazione trovata dal metodo di bisezione, fino ad ottenere la massima accuratezza possibile.

### Svolgimento

Il metodo ibrido combina:

- bisezione, che garantisce convergenza globale ma solo lineare;
- Newton, che converge quadraticamente se il punto iniziale è abbastanza vicino alla radice.

Si usa quindi la bisezione finché l'intervallo ha ampiezza minore di $\delta$; poi si avvia Newton dall'ultima approssimazione ottenuta, cercando la precisione di macchina.

```
function [x, it_bis, it_new] = metodoIbrido(f, f1, a, b, delta)
% Function [x, it_bis, it_new] = metodoIbrido(f, f1, a, b, delta)
%
% Metodo ibrido per la ricerca degli zeri di una funzione: inizialmente
% utilizza il metodo di bisezione e, quando l'ampiezza dell'intervallo
% diventa minore di delta, passa al metodo di Newton per ottenere la
% massima accuratezza possibile.
%
% Input:
%   f, f1 - function che implementano la funzione e la sua derivata;
%   a, b  - estremi dell'intervallo iniziale;
%   delta - ampiezza dell'intervallo per lo switch a Newton.
%
```

```
% Output:
%   x      - approssimazione della radice;
%   it_bis - numero di iterazioni di bisezione effettuate;
%   it_new - numero di iterazioni di Newton effettuate.
%
% Rel. 07-03-2026.

% gestione input e valori di default
if nargin < 5
    error('Numero di argomenti insufficiente.');
end

if a >= b
    error('Intervallo [a, b] non valido.');
end

if delta <= 0
    error('Parametro delta deve essere positivo.');
end

% inizializzazione
it_bis = 0;
it_new = 0;

% valutazione iniziale e controllo dei segni
fa = feval(f, a);
fb = feval(f, b);

if fa == 0
    x = a; return;
elseif fb == 0
    x = b; return;
elseif fa * fb > 0
    error('Condizione f(a)*f(b) < 0 non soddisfatta.');
end

% --- FASE 1: Metodo di Bisezione ---
% prosegue finche' l'ampiezza dell'intervallo e' >= delta
while (b - a) >= delta
    it_bis = it_bis + 1;

    % calcolo punto medio
    x = a + (b - a) / 2;
    fx = feval(f, x);

    if fx == 0
        return; % radice esatta trovata con bisezione
```

```
    end

    % aggiornamento intervallo
    if fa * fx < 0
        b = x;
        % inutile aggiornare fb, non viene mai usato nei controlli del ciclo
        % fb = fx;
    else
        a = x;
        fa = fx;
    end
end

% l'ultima approssimazione trovata e' il punto medio dell'intervallo corrente
x = a + (b - a) / 2;

% --- FASE 2: Metodo di Newton ---
% si parte dall'ultimo valore x trovato dalla bisezione
% per la massima accuratezza si utilizza una tolleranza prossima a eps
tol_new = 10 * eps;
maxit_new = 100; % valore di sicurezza

for i = 1:maxit_new
    x0 = x;

    fx = feval(f, x);
    f1x = feval(f1, x);

    if fx == 0
        break;
    end

    if f1x == 0
        warning('Arresto Newton: derivata nulla.');
        break;
    end

    it_new = it_new + 1;
    x = x0 - fx / f1x;

    % criterio di arresto per massima accuratezza
    if abs(x - x0) <= tol_new * (1 + abs(x))
        break;
    end
end

end
```

# Esercizio 6

Tabulare il costo computazionale dei precedenti metodi, in termini di valutazioni di funzione, per determinare lo zero della funzione

$$f(x) = \sin(10x) + \arctan(10x)$$

con la massima accuratezza possibile. Utilizzare $x_0 = 10$ per il metodo di Newton, e l'intervallo iniziale $[-10.1, 10]$ per il metodo di bisezione ed il metodo ibrido. Per quest'ultimo, utilizzare $\delta = 10^{-1}$ come valore di *switch* tra i due metodi. Commentare i risultati ottenuti.

## Svolgimento

### Analisi teorica

La funzione $f(x) = \sin(10x) + \arctan(10x)$ si annulla esattamente nell'origine ($x^* = 0$). Il termine trigonometrico la rende però oscillante e la derivata $f'(x) = 10\cos(10x) + \frac{10}{1+(10x)^2}$ cambia segno spesso; l'innesco $x_0 = 10$ è quindi sfavorevole per Newton.

### Costi computazionali

Con tolleranza dell'ordine di $10^{-15}$ si ottengono i risultati seguenti:

*Confronto del costo computazionale dei metodi.*

| Metodo | Iterazioni | Valutazioni Totali ($f$ o $f'$) | Esito |
|---|---|---|---|
| Bisezione | 54 | 56 | Successo |
| Newton ($x_0 = 10$) | 1000 (Max) | 2000 | Fallimento |
| Ibrido ($\delta = 0.1$) | 8 (Bis) + 3 (New) | 16 | Successo |

↳ per eseguire i test empirici e riprodurre i dati riportati in questa tabella, si faccia riferimento allo script MATLAB allegato denominato `test_metodi_zeri.m` .

### Commento

- **Bisezione**: converge sempre, ma solo linearmente; servono 54 iterazioni e 56 valutazioni di funzione.
- **Newton**: con $x_0 = 10$ non converge, perché le tangenti sono calcolate in una zona oscillante e allontanano gli iterati dalla radice.
- **Ibrido**: la bisezione porta l'intervallo sotto $\delta = 0.1$; da lì Newton converge in 3 iterazioni, con 16 valutazioni totali.

# Esercizio 7

Scrivere una function Matlab, con sintassi `x = triup(U,b)` , che risolva un sistema triangolare superiore $n \times n$, in cui la matrice dei coefficienti `U` è memorizzata, per colonne, in un vettore di lunghezza $\frac{n(n+1)}{2}$.

## Svolgimento

La soluzione implementa la sostituzione all'indietro per colonne, sfruttando la formula di indicizzazione $\text{idx}(i,j) = \frac{j(j-1)}{2} + i$ per accedere agli elementi di `U` memorizzati nel vettore compatto.

```
function x = triup(U, b)
% Function x = triup(U, b)
%
% Risolve un sistema triangolare superiore U*x = b di ordine n.
% La matrice U e' memorizzata per colonne in un vettore di
% lunghezza n*(n+1)/2.
%
% Input:
%   U - vettore contenente gli elementi della matrice triangolare superiore
%       (memorizzati per colonne);
%   b - vettore dei termini noti.
%
% Output:
%   x - vettore soluzione del sistema.
%
% Rel. 08-03-2026.

% ricavo la dimensione n dal vettore dei termini noti
n = length(b);

% controllo consistenza dimensioni
if length(U) ~= n*(n+1)/2
    error('Dimensioni incompatibili: U deve avere lunghezza n*(n+1)/2.');
end

U = U(:); % normalizza a vettore colonna per sicurezza
% inizializzazione soluzione
x = b(:); % normalizza a vettore colonna

% indice dell'ultimo elemento del vettore U (corrisponde a U_nn)
idx_end = n*(n+1)/2;

% back-substitution per colonne
for j = n : -1 : 1
```

```
    if U(idx_end) == 0
        error('Matrice singolare: elemento diagonale nullo in j=%d.', j);
    end

    % calcolo x(j)
    x(j) = x(j) / U(idx_end);

    % aggiornamento delle componenti precedenti
    if j > 1
        idx_start = idx_end - j + 1;
        x(1:j-1) = x(1:j-1) - U(idx_start:idx_end-1) * x(j);
        idx_end   = idx_start - 1;
    end
end

end
```

## Esercizio 8

Scrivere una function che implementi efficientemente la risoluzione di un sistema lineare mediante la fattorizzazione $LU$ con pivoting parziale. Validare la function su alcuni esempi significativi.

### Svolgimento

La function `LUSolve` realizza la fattorizzazione con pivoting parziale nella forma $PA = LU$ e memorizza i coefficienti in-place: la parte strettamente inferiore di `A` contiene i moltiplicatori di `L`, quella triangolare superiore i coefficienti di `U`.

```
function x = LUSolve(A, b)
% Function x = LUSolve(A, b)
%
% Risolve un sistema lineare A*x = b di ordine n usando la
% fattorizzazione LU con pivoting parziale.
%
% Input:
%   A - matrice dei coefficienti nxn;
%   b - vettore dei termini noti nx1.
%
% Output:
%   x - vettore soluzione del sistema nx1.
%
```

```
% Rel. 08-03-2026.

[m, n] = size(A);

% controlli di consistenza
if m ~= n
    error('La matrice A deve essere quadrata.');
end

if length(b) ~= n
    error('Dimensioni incompatibili tra A e b.');
end

% normalizzazione di b a vettore colonna
b = b(:);

% fattorizzazione LU con pivoting parziale (in-place)
p = 1:n;
tol = n * eps * norm(A, 1);

for i = 1:n
    % ricerca del pivot sulla colonna i
    [mi, ki] = max(abs(A(i:n, i)));
    ki = ki + i - 1;

    if mi <= tol
        error('Matrice singolare o quasi singolare.');
    end

    % scambio righe se necessario
    if ki > i
        A([i, ki], :) = A([ki, i], :);
        p([i, ki]) = p([ki, i]);
    end

    % eliminazione gaussiana
    if i < n
        A(i+1:n, i) = A(i+1:n, i) / A(i, i);
        A(i+1:n, i+1:n) = A(i+1:n, i+1:n) - A(i+1:n, i) * A(i, i+1:n);
    end
end

% risoluzione sistema PAx = Pb -> LUx = Pb
% risoluzione Ly = Pb (forward substitution)
y = b(p);
for i = 1:n-1
    y(i+1:n) = y(i+1:n) - A(i+1:n, i) * y(i);
```

```
end

% risoluzione Ux = y (backward substitution)
x = y;
for i = n:-1:1
    x(i) = x(i) / A(i, i);
    x(1:i-1) = x(1:i-1) - A(1:i-1, i) * x(i);
end

end
```

Dopo la fattorizzazione, la soluzione si ottiene risolvendo i due sistemi triangolari associati a `L` e `U`. Seguono alcuni test significativi:

1. **Sistema ben condizionato**: su un sistema quadrato non singolare, il vettore $x$ coincide con la soluzione restituita da MATLAB tramite `\`, con errore dell'ordine della precisione di macchina.
2. **Sistema che richiede pivoting**: con un elemento $(1,1)$ nullo, la funzione seleziona il pivot di modulo massimo, scambia le righe necessarie e calcola una soluzione coerente con quella di MATLAB.
3. **Matrice singolare (gestione errori)**: con righe linearmente dipendenti, il controllo sul pivot rileva la singolarità numerica e restituisce l'errore `"Matrice singolare o quasi singolare."`.

↳ Per eseguire i test empirici e riprodurre i risultati riportati, si faccia riferimento allo script MATLAB allegato denominato `test_lusolve.m`.

## Esercizio 9

Al seguente link: https://drive.google.com/file/d/1Fp8xFhUONS2W-l4alRKmomRxoM1SeVny/view?usp=share_link

```
% codice fornito dal professore per l'esercizio 9

function [A,b] = linsis(n,k,simme)
%
%  [A,b] = linsis(n,k,simme)  Crea una matrice A nxn ed un termine noto b,
%                             in modo che la soluzione del sistema lineare
%                             A*x=b sia x = [1,1,...,1]^T.
%                             k non ve lo dico a cosa serve.
%                             Se simme è specificato, crea una matrice sdp.
```

```
%
%                                                    Rel. 2026-01-07.
sigma = pi^(-2*k);
rng(0);
[q1,~] = qr(rand(n));
if nargin==3
    q2 = q1';
    B  = 0;
else
    [q2,~] = qr(rand(n));
    B      = diag(ones(n-1,1),1);
end
A = q1*(diag([sigma 2/n:1/n:1])+B)*q2;
b = sum(A,2);
return
```

è reperibile la function `linsis.m`, che crea la matrice dei coefficienti ed il termine noto di un sistema lineare $n \times n$, la cui soluzione è il vettore con tutte le componenti uguali ad 1. Utilizzare la function del precedente esercizio per risolvere i sistemi lineari ottenuti con le seguenti istruzioni:

```
n = 10;
for k = 1:15
    [A,b] = linsis(n,k);
end
```

Rappresentare *convenientemente* l'andamento dell'errore, rispetto ad `k`, e spiegare *esaurientemente* i risultati ottenuti.

## Svolgimento

Misuriamo l'accuratezza con l'errore in norma infinito

$$e_k = \|x_k - x^*\|_\infty,$$

rappresentato in scala semilogaritmica rispetto a $k$, così da evidenziarne la crescita su più ordini di grandezza.

### Risultati numerici

I test empirici forniscono i seguenti risultati:

*Andamento dell'errore di* `LUSolve` *al variare di* $k$ *per* $n = 10$.

| k | $\sigma_k = \pi^{-2k}$ | $\kappa_2(A_k)$ | $\|x_k - 1\|_\infty$ |
|---|---|---|---|
| 1 | $1.0132 \cdot 10^{-1}$ | $9.2774 \cdot 10^{3}$ | $2.5313 \cdot 10^{-13}$ |

| | | | |
|---|---|---|---|
| 2 | $1.0266 \cdot 10^{-2}$ | $9.1082 \cdot 10^{4}$ | $5.0440 \cdot 10^{-12}$ |
| 3 | $1.0402 \cdot 10^{-3}$ | $8.9890 \cdot 10^{5}$ | $2.2374 \cdot 10^{-10}$ |
| 4 | $1.0539 \cdot 10^{-4}$ | $8.8717 \cdot 10^{6}$ | $1.4721 \cdot 10^{-9}$ |
| 5 | $1.0678 \cdot 10^{-5}$ | $8.7561 \cdot 10^{7}$ | $4.8428 \cdot 10^{-9}$ |
| 6 | $1.0819 \cdot 10^{-6}$ | $8.6419 \cdot 10^{8}$ | $1.4339 \cdot 10^{-7}$ |
| 7 | $1.0962 \cdot 10^{-7}$ | $8.5292 \cdot 10^{9}$ | $2.3587 \cdot 10^{-7}$ |
| 8 | $1.1107 \cdot 10^{-8}$ | $8.4180 \cdot 10^{10}$ | $1.0476 \cdot 10^{-5}$ |
| 9 | $1.1254 \cdot 10^{-9}$ | $8.3084 \cdot 10^{11}$ | $3.4464 \cdot 10^{-5}$ |
| 10 | $1.1403 \cdot 10^{-10}$ | $8.2068 \cdot 10^{12}$ | $2.2674 \cdot 10^{-4}$ |
| 11 | $1.1553 \cdot 10^{-11}$ | $8.1185 \cdot 10^{13}$ | $2.2438 \cdot 10^{-3}$ |
| 12 | $1.1706 \cdot 10^{-12}$ | $8.3345 \cdot 10^{14}$ | $5.3792 \cdot 10^{-2}$ |
| 13 | $1.1861 \cdot 10^{-13}$ | $1.4299 \cdot 10^{16}$ | arresto della routine |
| 14 | $1.2017 \cdot 10^{-14}$ | $4.2866 \cdot 10^{16}$ | arresto della routine |
| 15 | $1.2176 \cdot 10^{-15}$ | $2.3192 \cdot 10^{16}$ | arresto della routine |

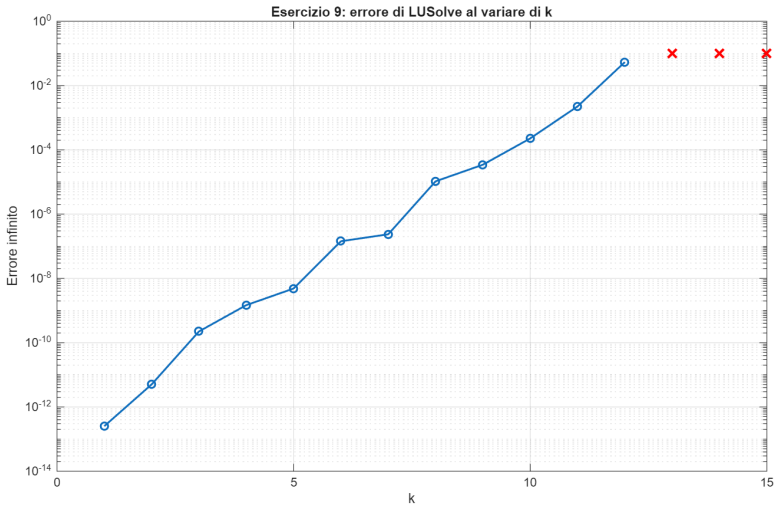



Figura 1: Grafico semilogaritmico dell'errore in norma infinito rispetto a $k$.

⚠ Per $k = 13, 14, 15$ la function `LUSolve` si interrompe con il messaggio `"Matrice singolare o quasi singolare."`. Questo comportamento non contraddice la costruzione teorica del sistema: segnala che, in aritmetica finita, i pivot diventano confrontabili con la soglia di sicurezza usata nel codice.

### Commento dei risultati

I dati evidenziano tre fenomeni.

- **Per valori piccoli di $k$** la matrice è ancora moderatamente ben condizionata e l'errore resta vicino alla precisione di macchina; per $k = 1$ è circa $2.5 \cdot 10^{-13}$.
- **All'aumentare di $k$** il parametro $\sigma_k = \pi^{-2k}$ si riduce di circa un fattore $\pi^2$ a ogni passo, mentre il numero di condizionamento cresce approssimativamente come $\frac{1}{\sigma_k}$. Nei primi

dodici casi il prodotto $\kappa_2(A_k)\sigma_k$ resta quasi costante, quindi il peggioramento è governato dal primo elemento diagonale.

- **L'errore cresce rapidamente**: quando $\kappa_2(A_k)$ arriva a circa $10^{15}$, gli errori di arrotondamento vengono fortemente amplificati. È il comportamento previsto per un problema mal condizionato.

Per $k = 12$ si osserva infatti un salto netto dell'errore a circa $5 \cdot 10^{-2}$. Da $k = 13$ in poi la matrice entra in una zona numericamente quasi singolare: la soglia

$$\text{tol} = n \cdot \text{eps} \cdot \|A\|_1$$

inserita in `LUSolve` intercetta pivot troppo piccoli e interrompe opportunamente l'algoritmo.

Il risultato distingue quindi il condizionamento del problema, che peggiora rapidamente con $k$, dall'accuratezza della soluzione, che decade fino al fallimento pratico della risoluzione numerica.

↳ Per riprodurre tabella e grafico riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es09.m` .

## Esercizio 10

Scrivere una function che implementi efficientemente la risoluzione di un sistema lineare, con matrice sdp, mediante la fattorizzazione $LDL^T$. Validare la function su alcuni esempi significativi.

### Svolgimento

La funzione `LDLSolve` esegue la fattorizzazione in-place per ridurre l'uso di memoria.

```
function x = LDLSolve(A, b)
% Function x = LDLSolve(A, b)
%
% Risolve un sistema lineare Ax = b con A s.d.p.
% mediante la fattorizzazione L D L^T.
% La funzione implementa la fattorizzazione in-place, sovrascrivendo
% la parte triangolare inferiore di A con il fattore L (parte strettamente
% inferiore, essendo la diagonale unitaria) e la diagonale di A con
% il fattore D.
%
% Input:
%   A - matrice simmetrica e definita positiva nxn;
```

```
%   b - vettore dei termini noti nx1.
%
% Output:
%   x - vettore soluzione nx1.
%
% Rel. 14-03-2026.

[m, n] = size(A);

% controlli di consistenza
if m ~= n || length(b) ~= n
    error('Dimensioni incompatibili o matrice non quadrata.');
end

% normalizzazione di b a vettore colonna
b = b(:);

% controllo simmetria
if norm(A - A', 'inf') > 1e-12 * norm(A, 'inf')
    error('La matrice A deve essere simmetrica.');
end

% fattorizzazione L D L^T in-place
d = zeros(n, 1);
for j = 1:n
    % v_i = L_{ji} * D_{ii} per i = 1...j-1
    % (vettore vuoto per j=1, il prodotto seguente vale 0)
    v = A(j, 1:j-1)' .* d(1:j-1);

    % D_{jj} = A_{jj} - L_{j, 1:j-1} * v
    A(j, j) = A(j, j) - A(j, 1:j-1) * v;

    if A(j, j) <= 0
        error('La matrice A non è definita positiva (d_%d = %g).', j, A(j,j));
    end

    % salvataggio elemento diagonale nel vettore d
    d(j) = A(j, j);

    % L_{j+1:n, j} = (A_{j+1:n, j} - A_{j+1:n, 1:j-1} * v) / D_{jj}
    if j < n
        A(j+1:n, j) = (A(j+1:n, j) - A(j+1:n, 1:j-1) * v) / d(j);
    end
end

% risoluzione sistema L D L^T x = b
```

```
% risoluzione L z = b (forward substitution)
z = b;
for j = 1:n-1
    z(j+1:n) = z(j+1:n) - A(j+1:n, j) * z(j);
end

% risoluzione D y = z (scaling)
y = z ./ d;

% risoluzione L^T x = y (backward substitution)
x = y;
for j = n:-1:2
    x(1:j-1) = x(1:j-1) - A(j, 1:j-1)' * x(j);
end

end
```

Seguono alcuni test per verificare accuratezza e gestione degli errori:

1. **Matrice simmetrica e definita positiva**: con una matrice s.d.p. casuale, ad esempio $A = X^T X$ con `X` non singolare, la soluzione coincide con quella ottenuta tramite `\`, con errore relativo dell'ordine di $10^{\{-16\}}$.
2. **Matrice non definita positiva**: con una matrice simmetrica ma non definita positiva, l'algoritmo rileva un elemento non positivo nel fattore $D$ e restituisce il messaggio `La matrice A non è definita positiva`.
3. **Matrice non simmetrica**: se $A \neq A^T$, il controllo iniziale in norma infinito rileva la discrepanza e segnala `La matrice A deve essere simmetrica`.

↳ Lo script MATLAB utilizzato per la validazione della funzione `LDLSolve` è `test_ldlsolve.m`

## Esercizio 11

Utilizzando la function `linsis.m` su menzionata, utilizzare la function del precedente esercizio per risolvere i sistemi lineari, con matrice sdp, ottenuti con le seguenti istruzioni:

```
n = 10;
for k = 1:15
    [A,b] = linsis(n,k,1);
end
```

Rappresentare *convenientemente* l'andamento dell'errore, rispetto ad `k`, e spiegare *esaurientemente* i risultati ottenuti.

## Svolgimento

Anche qui la soluzione esatta è

$$x^* = (1, ..., 1)^T,$$

e l'accuratezza si misura con l'errore in norma infinito

$$e_k = \|x_k - x^*\|_\infty.$$

Poiché l'errore varia su più ordini di grandezza, conviene rappresentare $e_k$ in scala semilogaritmica rispetto a $k$.

### Risultati numerici

Applicando `LDLSolve` ai quindici sistemi richiesti si ottengono i valori seguenti.

*Andamento dell'errore di `LDLSolve` al variare di $k$ per $n = 10$.*

| k | $\sigma_k = \pi^{-2k}$ | $\kappa_2(A_k)$ | $\|x_k - 1\|_\infty$ |
|---|---|---|---|
| 1 | $1.0132 \cdot 10^{-1}$ | 9.8696 | $5.5511 \cdot 10^{-16}$ |
| 2 | $1.0266 \cdot 10^{-2}$ | $9.7409 \cdot 10^{1}$ | $3.7748 \cdot 10^{-15}$ |
| 3 | $1.0402 \cdot 10^{-3}$ | $9.6139 \cdot 10^{2}$ | $4.5297 \cdot 10^{-14}$ |
| 4 | $1.0539 \cdot 10^{-4}$ | $9.4885 \cdot 10^{3}$ | $4.6407 \cdot 10^{-14}$ |
| 5 | $1.0678 \cdot 10^{-5}$ | $9.3648 \cdot 10^{4}$ | $6.6367 \cdot 10^{-12}$ |
| 6 | $1.0819 \cdot 10^{-6}$ | $9.2427 \cdot 10^{5}$ | $5.4831 \cdot 10^{-12}$ |
| 7 | $1.0962 \cdot 10^{-7}$ | $9.1222 \cdot 10^{6}$ | $4.6703 \cdot 10^{-11}$ |
| 8 | $1.1107 \cdot 10^{-8}$ | $9.0032 \cdot 10^{7}$ | $2.7144 \cdot 10^{-9}$ |
| 9 | $1.1254 \cdot 10^{-9}$ | $8.8858 \cdot 10^{8}$ | $4.8598 \cdot 10^{-8}$ |
| 10 | $1.1403 \cdot 10^{-10}$ | $8.7700 \cdot 10^{9}$ | $5.7300 \cdot 10^{-7}$ |
| 11 | $1.1553 \cdot 10^{-11}$ | $8.6556 \cdot 10^{10}$ | $2.8839 \cdot 10^{-7}$ |
| 12 | $1.1706 \cdot 10^{-12}$ | $8.5429 \cdot 10^{11}$ | $3.6724 \cdot 10^{-5}$ |
| 13 | $1.1861 \cdot 10^{-13}$ | $8.4349 \cdot 10^{12}$ | $2.7192 \cdot 10^{-4}$ |
| 14 | $1.2017 \cdot 10^{-14}$ | $8.3045 \cdot 10^{13}$ | $2.5704 \cdot 10^{-3}$ |
| 15 | $1.2176 \cdot 10^{-15}$ | $7.7556 \cdot 10^{14}$ | $9.7005 \cdot 10^{-3}$ |

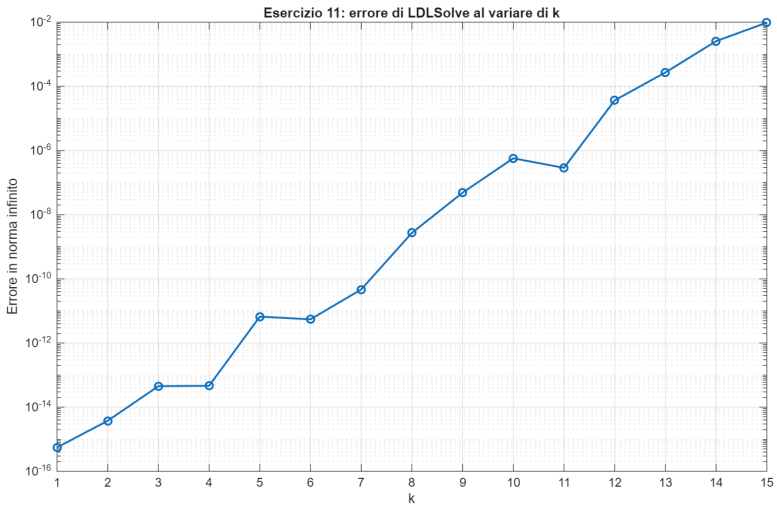



Figura 2: Grafico semilogaritmico dell'errore in norma infinito rispetto a $k$ nel caso s.d.p..

### Commento dei risultati

L'errore cresce con $k$: $\sigma_k = \pi^{-2k}$ diminuisce, il numero di condizione aumenta e gli arrotondamenti vengono amplificati. Per i primi valori l'errore resta vicino alla precisione di macchina, mentre per $k = 15$ arriva a circa $10^{-2}$.

A differenza del caso generale dell'esercizio 9, `LDLSolve` risolve tutti i sistemi: la struttura s.d.p. rende appropriata la fattorizzazione $LDL^T$.

↳ Per riprodurre tabella e grafico riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es11.m` .

## Esercizio 12

Dato un generico sistema lineare sovradeterminato a rango massimo,

$$Ax = b, \quad A \in \mathbb{R}^{m\times n}, \quad m > n = \text{rank}(A),$$

ed indicato, al solito, con $r := Ax - b \in \mathbb{R}^m$ il corrispondente vettore residuo, scrivere una function Matlab che risolva il precedente problema nel senso dei minimi quadrati *pesati*. Ovvero, minimizzando la seguente norma pesata:

$$\|r\|_\omega^2 = \sum_{i=1}^{m} \omega_i r_i^2,$$

dove $\omega_1, ..., \omega_m$ sono pesi positivi assegnati.

### Svolgimento

Introduciamo la matrice diagonale dei pesi

$$W = \text{diag}(\omega_1, ..., \omega_m),$$

simmetrica definita positiva perché tutti i pesi sono strettamente positivi. La sua radice quadrata è

$$W^{\frac{1}{2}} = \text{diag}(\sqrt{\omega_1}, ..., \sqrt{\omega_m}),$$

e consente di ricondurre il problema pesato a uno standard. Infatti

$$\left(\sqrt{\omega_i} r_i\right)^2 = \omega_i r_i^2.$$

Perciò la quantità da minimizzare si può riscrivere come

$$\|r\|_\omega^2 = r^T W r = \|W^{\frac{1}{2}} r\|_2^2 = \|W^{\frac{1}{2}}(Ax - b)\|_2^2.$$

Ponendo quindi

$$A_w = W^{\frac{1}{2}} A \quad \text{e} \quad b_w = W^{\frac{1}{2}} b,$$

il problema dei minimi quadrati pesati diventa un ordinario problema ai minimi quadrati:

$$\min_{x \in \mathbb{R}^n} \|A_w x - b_w\|_2.$$

Poiché $W^{\frac{1}{2}}$ è invertibile e $A$ ha rango colonna massimo, anche $A_w$ ha rango $n$. Si calcola quindi una fattorizzazione `QR` ridotta mediante riflessioni di Householder, senza assemblare esplicitamente $Q$,

$$A_w = QR, \quad Q \in \mathbb{R}^{m \times n}, \quad Q^T Q = I_n, \quad R \in \mathbb{R}^{n \times n},$$

con $R$ triangolare superiore e non singolare. Il minimizzatore è l'unica soluzione del sistema triangolare

$$Rx = Q^T b_w.$$

```
function x = WeightedLSSolve(A, b, omega)
% Function x = WeightedLSSolve(A, b, omega)
%
% Risolve un problema di minimi quadrati pesati
% con A matrice m x n a rango colonna massimo e m >= n,
% mediante una fattorizzazione QR del sistema scalato
% calcolata con riflessioni di Householder.
%
% Input:
%   A     - matrice dei coefficienti m x n;
%   b     - vettore dei dati m x 1;
%   omega - vettore dei pesi positivi m x 1.
%
% Output:
%   x     - soluzione di minimi quadrati pesati n x 1.
% Rel. 14-03-2026.
```

```
[m, n] = size(A);

% controlli di consistenza
if m < n
    error('Il sistema deve essere sovradeterminato o quadrato.');
end

if length(b) ~= m || length(omega) ~= m
    error('Dimensioni incompatibili tra A, b e omega.');
end

% normalizzazione a vettori colonna
b = b(:);
omega = omega(:);

% controllo positivita' dei pesi
if any(omega <= 0)
    error('I pesi devono essere strettamente positivi.');
end

% costruzione implicita del sistema scalato:
% Aw = W^(1/2) A, bw = W^(1/2) b
d = sqrt(omega);
A = A .* d;
b = d .* b;

% fattorizzazione QR ridotta tramite riflessioni di Householder
tol = max(m, n) * eps * norm(A, 'inf');

for k = 1:n
    xk = A(k:m, k);
    sigma = norm(xk);

    if sigma <= tol
        error('La matrice A non ha rango colonna massimo.');
    end

    if xk(1) >= 0
        alpha = -sigma;
    else
        alpha = sigma;
    end

    v = xk;
    v(1) = v(1) - alpha;
    beta = v' * v;
```

```
    A(k:m, k:n) = A(k:m, k:n) - (2 / beta) * v * (v' * A(k:m, k:n));
    b(k:m) = b(k:m) - (2 / beta) * v * (v' * b(k:m));

    A(k, k) = alpha;
    A(k+1:m, k) = 0;
end

R = triu(A(1:n, 1:n));
y = b(1:n);

% risoluzione Rx = y (backward substitution)
x = y;
for i = n:-1:1
    x(i) = x(i) / R(i, i);
    x(1:i-1) = x(1:i-1) - R(1:i-1, i) * x(i);
end

end
```

## Esercizio 13

Siano assegnati:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 0 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}.$$

Risolvere il sistema $A\boldsymbol{x} = \boldsymbol{b}$ nel senso dei minimi quadrati pesati, cioè minimizzando (1), nei seguenti casi:

- $\omega_i = 1, \quad i = 1, ..., 4$ (minimi quadrati standard);
- $\omega_i = \frac{(5-i)^2}{9}, \quad i = 1, ..., 4;$
- $\omega_i = \frac{i^2}{9}, \quad i = 1, ..., 4;$
- $\omega_i = \sqrt{2}, \quad i = 1, ..., 4.$

Riportare la norma (il quadrato della norma pesata) (1) del corrispondente residuo per tutti i casi. Commentare i risultati ottenuti.

(1) Il quadrato della norma pesata del residuo: $\Phi(\boldsymbol{x}) = \sum_{i=1}^{4} \omega_i (\boldsymbol{b}_i - (A\boldsymbol{x})_i)^2$.

### Svolgimento

Indichiamo con

$$\boldsymbol{r}(\boldsymbol{x}) = \boldsymbol{b} - A\boldsymbol{x}$$

il residuo associato a $\boldsymbol{x}$. La quantità da minimizzare è

$$\Phi(\boldsymbol{x}) = \sum_{i=1}^{4} \omega_i r_{i(\boldsymbol{x})}^2,$$

cioè il quadrato della norma pesata del residuo. Poiché i pesi sono positivi, $\Phi(\boldsymbol{x}) \geq 0$ e il valore minimo possibile è 0.

In questo caso il sistema è compatibile. Infatti la sottomatrice $3 \times 3$ ottenuta scegliendo le righe $1, 2, 4$ di $A$ ha determinante

$$36 \neq 0,$$

quindi $A$ ha rango colonna massimo, cioè $\text{rank}(A) = 3$. Ricaviamo esplicitamente la soluzione. Le quattro equazioni sono

$$x_1 + 2x_2 + 3x_3 = 1, \quad 4x_1 + 5x_2 + 6x_3 = 2,$$

$$7x_1 + 8x_2 + 9x_3 = 3, \quad 10x_1 + 11x_2 = 4.$$

Sottraendo la prima equazione dalla seconda e la seconda dalla terza si ottiene in entrambi i casi

$$x_1 + x_2 + x_3 = \frac{1}{3}.$$

Sottraendo invece la terza equazione dalla quarta si ha

$$x_1 + x_2 - 3x_3 = \frac{1}{3}.$$

Facendo ora la differenza tra le ultime due relazioni segue

$$4x_3 = 0, \quad \text{quindi} \quad x_3 = 0.$$

Sostituendo in $x_1 + x_2 + x_3 = \frac{1}{3}$ si ricava

$$x_1 + x_2 = \frac{1}{3}.$$

Insieme alla prima equazione,

$$x_1 + 2x_2 = 1,$$

si ottiene

$$x_2 = \frac{2}{3}, \quad x_1 = -\frac{1}{3}.$$

Dunque la soluzione esatta è

$$\boldsymbol{x}^* = \begin{pmatrix} -\frac{1}{3} \\ \frac{2}{3} \\ 0 \end{pmatrix}.$$

Si verifica infatti immediatamente che

$$A\boldsymbol{x}^* = \boldsymbol{b},$$

perciò il residuo corrispondente è

$$\boldsymbol{r}(\boldsymbol{x}^*) = \boldsymbol{b} - A\boldsymbol{x}^* = \boldsymbol{0}.$$

e dunque, per **ogni** scelta di pesi positivi,

$$\Phi(\boldsymbol{x}^*) = \sum_{i=1}^{4} \omega_i (r_i^*)^2 = 0.$$

Quindi tutti e quattro i casi hanno la stessa soluzione di minimi quadrati, coincidente con la soluzione esatta del sistema.

### Risultati numerici

Applicando `WeightedLSSolve` si ottengono i seguenti risultati numerici.

*Soluzioni di minimi quadrati pesati per i quattro insiemi di pesi assegnati.*

| Caso | Pesi | $\boldsymbol{x}$ calcolata | $\Phi(\boldsymbol{x})$ |
|---|---|---|---|
| 1 | $\omega_i = 1$ | $\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$ | $2.4652 \cdot 10^{-31}$ |
| 2 | $\omega_i = \frac{(5-i)^2}{9}$ | $\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$ | $5.0947 \cdot 10^{-31}$ |
| 3 | $\omega_i = \frac{i^2}{9}$ | $\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$ | $2.4981 \cdot 10^{-30}$ |
| 4 | $\omega_i = \sqrt{2}$ | $\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$ | $5.5781 \cdot 10^{-31}$ |

I valori in tabella sono dell'ordine di $10^{-30}$, quindi compatibili con un residuo nullo a meno degli arrotondamenti. In aritmetica esatta si avrebbe

$$\boldsymbol{r}(\boldsymbol{x}^*) = \boldsymbol{0} \quad \text{e quindi} \quad \Phi(\boldsymbol{x}^*) = 0$$

in tutti e quattro i casi.

↳ Per riprodurre i valori numerici riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es13.m` .

## Esercizio 14

Scrivere una function che implementi efficientemente il metodo di Newton per risolvere sistemi di equazioni nonlineari. Utilizzare le function sviluppate in precedenza come risolutori per i sistemi lineari coinvolti.

### Svolgimento

Sia assegnato un sistema nonlineare

$$\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}, \quad \boldsymbol{F} : \mathbb{R}^n \to \mathbb{R}^n.$$

Linearizzando $\boldsymbol{F}$ attorno all'iterato corrente $\boldsymbol{x}^k$ si ottiene

$$\boldsymbol{F}(\boldsymbol{x}^k + \Delta\boldsymbol{x}^k) \approx \boldsymbol{F}(\boldsymbol{x}^k) + J_{\boldsymbol{F}}(\boldsymbol{x}^k)\Delta\boldsymbol{x}^k,$$

dove $J_{\boldsymbol{F}}(\boldsymbol{x}^k)$ è la matrice Jacobiana. Imporre l'annullamento del termine lineare porta al sistema di Newton

$$J_{\boldsymbol{F}}(\boldsymbol{x}^k)\Delta\boldsymbol{x}^k = -\boldsymbol{F}(\boldsymbol{x}^k),$$

seguito dall'aggiornamento

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \Delta\boldsymbol{x}^k.$$

Per un'implementazione efficiente conviene:

- non calcolare mai esplicitamente l'inversa della Jacobiana;
- far restituire a `fun` sia $\boldsymbol{F}(\boldsymbol{x})$ sia $J_{\boldsymbol{F}}(\boldsymbol{x})$, così eventuali quantità intermedie possono essere riutilizzate;
- risolvere il sistema lineare con una routine già sviluppata, usando `LUSolve` come default e lasciando la possibilità di passare un altro risolutore, ad esempio `LDLSolve`, quando la Jacobiana abbia struttura favorevole.

Il criterio di arresto è l'estensione vettoriale del criterio misto assoluto/relativo già usato per Newton scalare:

$$\left\| \Delta\boldsymbol{x}^k \frac{\cdot}{1 + |\boldsymbol{x}^{k+1}|} \right\|_\infty \leq \text{tol}.$$

Il denominatore contiene l'approssimazione corrente $\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \Delta\boldsymbol{x}^k$, così il passo è valutato rispetto alla scala delle componenti della soluzione.

```
function [x, it] = metodoNewtonSistemi(fun, x0, tol, maxit, solverLineare)
% Function [x, it] = metodoNewtonSistemi(fun, x0, tol, maxit, solverLineare)
%
% Metodo di Newton per risolvere sistemi nonlineari F(x) = 0.
% La function fun deve restituire sia F(x) sia la Jacobiana J(x)
% mediante la sintassi [Fx, Jx] = fun(x).
%
% Input:
%   fun           - function handle che restituisce [F(x), J(x)];
%   x0            - punto iniziale;
%   tol           - tolleranza di arresto (default 10*eps);
%   maxit         - numero massimo di iterazioni (default 1000);
%   solverLineare - risolutore del sistema J(x)*dx = -F(x)
%                   (default @LUSolve).
%
% Output:
%   x             - approssimazione della soluzione;
%   it            - numero di iterazioni effettuate.
%
% Rel. 21-03-2026.
```

```
% gestione degli argomenti di input e valori di default
if nargin < 2
    error('Numero di argomenti insufficiente.');
end
if nargin < 5 || isempty(solverLineare)
    solverLineare = @LUSolve;
end
if nargin < 4 || isempty(maxit)
    maxit = 1000;
end
if nargin < 3 || isempty(tol)
    tol = 10 * eps;
end

% controllo validita' input
if ~isa(fun, 'function_handle')
    error('Il primo argomento deve essere una function.');
end
if ~isa(solverLineare, 'function_handle')
    error('Il risolutore lineare deve essere una function.');
end
if tol <= 0
    error('Tolleranza non corretta: deve essere > 0.');
end
if maxit < 1 || maxit ~= fix(maxit)
    error('Numero massimo di iterazioni non corretto.');
end

% inizializzazione
x = x0(:);
it = 0;
flag = 0;

% ciclo iterativo di Newton
for k = 1:maxit
    [Fx, Jx] = feval(fun, x);
    Fx = Fx(:);
    [m, n] = size(Jx);

    if m ~= n || length(Fx) ~= n
        error('La function fun deve restituire F(x) e una Jacobiana quadrata
compatibili.');
    end

    % controllo se ho gia' trovato uno zero esatto
    if norm(Fx, inf) == 0
        flag = 1;
```

```
        break;
    end

    % passo di Newton: J(x_k) * dx = -F(x_k)
    dx = feval(solverLineare, Jx, -Fx);
    x_new = x + dx;
    it = k;

    % criterio di arresto misto assoluto/relativo:
    % nel denominatore compare l'approssimazione corrente x_{k+1}
    if norm(dx ./ (1 + abs(x_new)), inf) <= tol
        x = x_new;
        flag = 1;
        break;
    end

    x = x_new;
end

% avviso se non si e' raggiunta la convergenza
if flag == 0
    warning('Accuratezza non raggiunta entro il numero massimo di iterazioni.');
end

end
```

## Esercizio 15

Utilizzare la precedente function per risolvere il sistema nonlineare:

$$x_1^2 + x_2^2 = 5, \quad x_3^2 + x_4^2 = 5, \quad \cos(\pi x_1) + \cos(\pi x_3) = 0, \quad x_2^2 - x_4^2 = 0.$$

Quante iterazioni sono necessarie per ottenere la massima accuratezza possibile, partendo dai punti iniziali:

$$\boldsymbol{x}^0 = \begin{pmatrix} 0.4 \\ 2 \\ 0.4 \\ 2 \end{pmatrix}, \quad \boldsymbol{x}^0 = \begin{pmatrix} 0.4 \\ -2 \\ -0.4 \\ 2 \end{pmatrix}, \quad \boldsymbol{x}^0 = \begin{pmatrix} 1.4 \\ -2 \\ -1.4 \\ 2 \end{pmatrix}, \quad \boldsymbol{x}^0 = \begin{pmatrix} 1.4 \\ 2 \\ 1.4 \\ 2 \end{pmatrix}.$$

Riportare le soluzioni ottenute e commentare i risultati. Sarebbe possibile utilizzare il vettore nullo come punto iniziale (argomentare la risposta)?

### Svolgimento

Introduciamo il sistema in forma vettoriale

$$\boldsymbol{F}(\boldsymbol{x}) = \begin{pmatrix} x_1^2 + x_2^2 - 5 \\ x_3^2 + x_4^2 - 5 \\ \cos(\pi x_1) + \cos(\pi x_3) \\ x_2^2 - x_4^2 \end{pmatrix},$$

con Jacobiana

$$J_{\boldsymbol{F}}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 2x_2 & 0 & 0 \\ 0 & 0 & 2x_3 & 2x_4 \\ -\pi \sin(\pi x_1) & 0 & -\pi \sin(\pi x_3) & 0 \\ 0 & 2x_2 & 0 & -2x_4 \end{pmatrix}.$$

Ad ogni iterazione si risolve il sistema lineare

$$J_{\boldsymbol{F}}\big(\boldsymbol{x}^k\big)\Delta\boldsymbol{x}^k = -\boldsymbol{F}\big(\boldsymbol{x}^k\big)$$

e si aggiorna

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \Delta\boldsymbol{x}^k.$$

La Jacobiana non è in generale simmetrica, quindi si usa `LUSolve`, già default in `metodoNewtonSistemi`. Per la massima accuratezza si pone `tol = 10 * eps`.

Prima di riportare i risultati numerici conviene descrivere la struttura delle soluzioni esatte. Dalla quarta equazione segue

$$x_2^2 = x_4^2.$$

Sottraendo poi la seconda equazione dalla prima si ottiene

$$x_1^2 - x_3^2 = 0, \quad \text{cioe'} \quad x_3 = \pm x_1.$$

Poiché il coseno è pari, risulta

$$\cos(\pi x_3) = \cos(\pm \pi x_1) = \cos(\pi x_1),$$

e la terza equazione diventa

$$2\cos(\pi x_1) = 0, \quad \text{da cui} \quad x_1 = \frac{1}{2} + k, \quad k \in \mathbb{Z}.$$

Imporre anche $x_1^2 + x_2^2 = 5$ limita i valori ammissibili a

$$x_1 \in \left\{ \pm\frac{1}{2}, \pm\frac{3}{2} \right\}.$$

Ne segue che le soluzioni del sistema hanno necessariamente una delle forme

$$x_1 = \pm\frac{1}{2}, \quad x_2 = \pm\frac{\sqrt{19}}{2},$$

oppure

$$x_1 = \pm\frac{3}{2}, \quad x_2 = \pm\frac{\sqrt{11}}{2},$$

con inoltre $x_3 = \pm x_1$ e $x_4 = \pm x_2$.

### Risultati numerici

Applicando `metodoNewtonSistemi` ai quattro punti iniziali assegnati si ottengono i risultati seguenti.

*Soluzioni ottenute con `metodoNewtonSistemi` e `tol = 10 * eps` per i quattro inneschi assegnati.*

| Caso | $x^0$ | $x^*$ ottenuta | Iterazioni | $\|F(x^*)\|_\infty$ |
|---|---|---|---|---|
| 1 | $(0.4, 2, 0.4, 2)^T$ | $\left(\frac{1}{2}, \frac{\sqrt{19}}{2}, \frac{1}{2}, \frac{\sqrt{19}}{2}\right)^T$ | 5 | $8.8818 \cdot 10^{-16}$ |
| 2 | $(0.4, -2, -0.4, 2)^T$ | $\left(\frac{1}{2}, -\frac{\sqrt{19}}{2}, -\frac{1}{2}, \frac{\sqrt{19}}{2}\right)^T$ | 5 | $8.8818 \cdot 10^{-16}$ |
| 3 | $(1.4, -2, -1.4, 2)^T$ | $\left(\frac{3}{2}, -\frac{\sqrt{11}}{2}, -\frac{3}{2}, \frac{\sqrt{11}}{2}\right)^T$ | 5 | $3.6739 \cdot 10^{-16}$ |
| 4 | $(1.4, 2, 1.4, 2)^T$ | $\left(\frac{3}{2}, \frac{\sqrt{11}}{2}, \frac{3}{2}, \frac{\sqrt{11}}{2}\right)^T$ | 5 | $3.6739 \cdot 10^{-16}$ |

In tutti e quattro i casi Newton raggiunge la precisione di macchina in 5 iterazioni. I residui finali sono dell'ordine di $10^{-16}$, quindi compatibili con la massima accuratezza in doppia precisione.

I primi due inneschi convergono verso soluzioni con $|x_1| = |x_3| = \frac{1}{2}$, gli ultimi due verso soluzioni con $|x_1| = |x_3| = \frac{3}{2}$: il metodo seleziona la radice nel bacino di attrazione del punto iniziale. Il numero di iterazioni coincide perché tutti gli inneschi sono già abbastanza vicini a una radice semplice.

### Punto iniziale nullo

Non è possibile usare il vettore nullo come punto iniziale. Infatti

$$\boldsymbol{F}(\mathbf{0}) = \begin{pmatrix} -5 \\ -5 \\ 2 \\ 0 \end{pmatrix} \neq \mathbf{0},$$

mentre la Jacobiana nel punto nullo e'

$$J_{\boldsymbol{F}}(\mathbf{0}) = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Il primo passo di Newton richiederebbe quindi di risolvere il sistema

$$J_{\boldsymbol{F}}(\mathbf{0})\Delta\boldsymbol{x} = -\boldsymbol{F}(\mathbf{0}),$$

cioè un sistema con matrice nulla e termine noto non nullo, impossibile da risolvere. Per questo Newton per sistemi richiede che la Jacobiana sia invertibile nei punti in cui viene eseguito il passo.

↳ Per riprodurre i valori numerici riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es15.m` .

## Esercizio 16

Scrivere una function Matlab,

```
y = lagrange( xi, fi, x )
```

in cui `(xi,fi)` contengono i dati del problema e `x` è un vettore di punti a cui corrispondono le valutazioni del polinomio interpolante in forma di Lagrange restituite nel vettore `y` .

### Svolgimento

Non conviene costruire il polinomio nella base delle potenze: è sufficiente valutare, per ogni $i$, il polinomio fondamentale $L_{i,n}$ nei punti richiesti e accumulare il contributo $f_i L_{i,n}$.

```
function y = lagrange(xi, fi, x)
% Function y = lagrange(xi, fi, x)
%
% Valuta il polinomio interpolante nella forma di Lagrange
% associato ai dati (xi, fi) nei punti contenuti nel vettore x.
%
% Input:
%   xi - vettore delle ascisse di interpolazione;
%   fi - vettore dei valori della funzione nelle ascisse xi;
%   x  - vettore dei punti di valutazione.
%
% Output:
%   y  - valori del polinomio interpolante nei punti x.
%
% Rel. 14-05-2026.

% controlli di consistenza
if nargin < 3
    error('Numero di argomenti insufficiente.');
end
```

```
n = length(xi);

if length(fi) ~= n
    error('Dimensioni incompatibili tra xi e fi.');
end

if n == 0
    error('Il vettore xi non puo'' essere vuoto.');
end

if length(unique(xi)) ~= n
    error('Le ascisse di interpolazione devono essere distinte.');
end

y = zeros(size(x));

% valutazione del polinomio p(x) = sum_i fi_i * L_i(x)
for i = 1:n
    Li = ones(size(x));

    for j = 1:n
        if j ~= i
            Li = Li .* (x - xi(j)) / (xi(i) - xi(j));
        end
    end

    y = y + fi(i) * Li;
end

end
```

Il costo è $O(mn^2)$, dove $m$ è il numero di punti contenuti in `x` e $n+1$ il numero di dati di interpolazione.

## Esercizio 17

Scrivere una function Matlab,

```
y = newton( xi, fi, x )
```

che ha la stessa sintassi e funzione della function del precedente esercizio, ma con l'utilizzo della forma di Newton del polinomio interpolante.

### Svolgimento

La forma di Newton usa i coefficienti

$$a_i = f[x_0, ..., x_i], \quad i = 0, ..., n,$$

cioè le differenze divise sui primi $i+1$ nodi. Il polinomio interpolante si scrive quindi come

$$p_{n(x)} = \sum_{i=0}^{n} a_i \omega_{i(x)}, \quad \omega_0(x) = 1, \quad \omega_{i(x)} = \prod_{j=0}^{i-1} (x - x_j).$$

Calcolate le differenze divise, la valutazione si effettua con Horner modificato, evitando sia la base delle potenze sia la costruzione esplicita dei polinomi $\omega_i$.

```
function y = newton(xi, fi, x)
% Function y = newton(xi, fi, x)
%
% Valuta il polinomio interpolante nella forma di Newton
% associato ai dati (xi, fi) nei punti contenuti nel vettore x.
%
% Input:
%   xi - vettore delle ascisse di interpolazione;
%   fi - vettore dei valori della funzione nelle ascisse xi;
%   x  - vettore dei punti di valutazione.
%
% Output:
%   y  - valori del polinomio interpolante nei punti x.
%
% Rel. 14-05-2026.

% controlli di consistenza
if nargin < 3
    error('Numero di argomenti insufficiente.');
end

n = length(xi);

if length(fi) ~= n
    error('Dimensioni incompatibili tra xi e fi.');
end

if n == 0
    error('Il vettore xi non puo'' essere vuoto.');
end

if length(unique(xi)) ~= n
    error('Le ascisse di interpolazione devono essere distinte.');
end

xi = xi(:);
a = fi(:);
```

```
% calcolo in-place delle differenze divise
for j = 2:n
    for i = n:-1:j
        a(i) = (a(i) - a(i - 1)) / (xi(i) - xi(i - j + 1));
    end
end

% valutazione con Horner modificato per la base di Newton
y = a(n) * ones(size(x));

for i = n-1:-1:1
    y = y .* (x - xi(i)) + a(i);
end

end
```

Il calcolo delle differenze divise richiede $O(n^2)$ operazioni, mentre la valutazione in $m$ punti richiede $O(mn)$. Rispetto alla forma di Lagrange, quella di Newton è più comoda quando si aggiungono nodi, perché basta calcolare le nuove differenze divise.

## Esercizio 18

Scrivere una function Matlab,

```
[y,y1] = hermite( xi, fi, fi1, x )
```

in cui `(xi,fi,fi1)` contengono i dati del problema e `x` è un vettore di punti a cui corrispondono le valutazioni del polinomio interpolante di Hermite, e della sua derivata prima, rispettivamente restituite nei vettori `y` e `y1`.

### Svolgimento

Per costruire il polinomio di Hermite usiamo ancora la forma di Newton, ripetendo ogni nodo due volte. Se $z_{2i-1} = z_{2i} = x_i$, allora la differenza divisa relativa alla coppia di nodi coincidenti non si calcola con un quoziente, ma si pone uguale al dato sulla derivata:

$$f[z_{2i-1}, z_{2i}] = f'_i.$$

Ottenuti i coefficienti di Newton, polinomio e derivata prima si valutano con Horner modificato.

```
function [y, y1] = hermite(xi, fi, fi1, x)
% Function [y, y1] = hermite(xi, fi, fi1, x)
%
% Valuta il polinomio interpolante di Hermite associato ai dati
% (xi, fi, fi1) nei punti contenuti nel vettore x.
%
% Input:
%   xi  - vettore delle ascisse di interpolazione;
%   fi  - vettore dei valori della funzione nelle ascisse xi;
%   fi1 - vettore dei valori della derivata prima nelle ascisse xi;
%   x   - vettore dei punti di valutazione.
%
% Output:
%   y   - valori del polinomio interpolante nei punti x;
%   y1  - valori della derivata prima del polinomio nei punti x.
%
% Rel. 15-05-2026.

% controlli di consistenza
if nargin < 4
    error('Numero di argomenti insufficiente.');
end

n = length(xi);

if length(fi) ~= n || length(fi1) ~= n
    error('Dimensioni incompatibili tra xi, fi e fi1.');
end

if n == 0
    error('Il vettore xi non puo'' essere vuoto.');
end

if length(unique(xi)) ~= n
    error('Le ascisse di interpolazione devono essere distinte.');
end

xi = xi(:);
fi = fi(:);
fi1 = fi1(:);

N = 2 * n;
z = zeros(N, 1);
a = zeros(N, 1);

z(1:2:N) = xi;
z(2:2:N) = xi;
```

```
a(1:2:N) = fi;
a(2:2:N) = fi;

% calcolo in-place delle differenze divise con nodi ripetuti
for j = 2:N
    for i = N:-1:j
        if j == 2 && rem(i, 2) == 0
            a(i) = fi1(i / 2);
        else
            a(i) = (a(i) - a(i - 1)) / (z(i) - z(i - j + 1));
        end
    end
end

% valutazione simultanea di p e p' con Horner modificato
y = a(N) * ones(size(x));
y1 = zeros(size(x));

for i = N-1:-1:1
    y1 = y + (x - z(i)) .* y1;
    y = y .* (x - z(i)) + a(i);
end

end
```

La costruzione delle differenze divise richiede $O(n^2)$ operazioni, dove `n` è il numero di nodi passati alla function. La valutazione in $m$ punti richiede $O(mn)$ operazioni e produce direttamente sia `y` sia `y1`.

## Esercizio 19

Scrivere una function Matlab,

```
x = chebyshev( a, b, n )
```

che restituisca le ascisse di Chebyshev per il polinomio interpolante di grado $n$ sull'intervallo $[a, b]$.

### Svolgimento

Per un polinomio interpolante di grado $n$ servono $n + 1$ nodi. Si usano quindi gli zeri di $T_{n+1}$, trasportati linearmente da $[-1, 1]$ a $[a, b]$:

$$x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos\left((2i+1)\frac{\pi}{2(n+1)}\right), \quad i = 0, \ldots, n.$$

Nel codice si prende `i = n:-1:0`, così le ascisse sono restituite in ordine crescente.

```
function x = chebyshev(a, b, n)
% Function x = chebyshev(a, b, n)
%
% Restituisce le n + 1 ascisse di Chebyshev per il polinomio
% interpolante di grado n sull'intervallo [a, b].
%
% Input:
%   a, b - estremi dell'intervallo di interpolazione;
%   n    - grado del polinomio interpolante.
%
% Output:
%   x    - vettore riga delle ascisse di Chebyshev in [a, b].
%
% Rel. 15-05-2026.

% controlli di consistenza
if nargin < 3
    error('Numero di argomenti insufficiente.');
end

if a >= b
    error('Intervallo [a, b] non valido.');
end

if n < 0 || n ~= fix(n)
    error('Il grado n deve essere un intero non negativo.');
end

i = n:-1:0;

% zeri di T_(n+1) su [-1, 1] e trasformazione affine in [a, b]
x = cos((2 * i + 1) * pi / (2 * (n + 1)));
x = (a + b) / 2 + (b - a) / 2 * x;

return
end
```

## Esercizio 20

Graficare l'errore massimo (calcolato come il massimo di 10001 punti equispaziati nell'intervallo) per i polinomi interpolanti degli esercizi 16-18, relative alla funzione di Runge,

$$f(x) = (1 + x^2)^{-1}, \quad x \in [-10, 10],$$

utilizzando $n + 1$ ascisse equispaziate, con $n = 10, 20, 30, ..., 140$, per i polinomi interpolanti in 16-17, e $n = 10, 20, 30, ..., 70$, per il polinomio interpolante di Hermite. Per quest'ultimo polinomio, graficare anche l'errore della derivata prima. Utilizzare il modo più appropriato per graficare i risultati. Commentare i risultati ottenuti.

## Svolgimento

L'errore massimo è stato stimato, come richiesto, su 10001 punti equispaziati dell'intervallo $[-10, 10]$. Li indichiamo con

$$t_j = -10 + j \cdot \left(\frac{20}{10000}\right), \quad j = 0, ..., 10000,$$

dove 20 è la lunghezza dell'intervallo. Su questi punti si pone

$$E_n = \max_{0 \le j \le 10000} |f(t_j) - p_{n(t_j)}|.$$

Per il polinomio di Hermite è stato calcolato anche

$$E'_n = \max_{0 \le j \le 10000} |f'(t_j) - p'_{n(t_j)}|, \quad f'(x) = -\frac{2x}{(1 + x^2)^2}.$$

Poiché gli errori variano su molti ordini di grandezza, la rappresentazione più adatta è quella semilogaritmica.

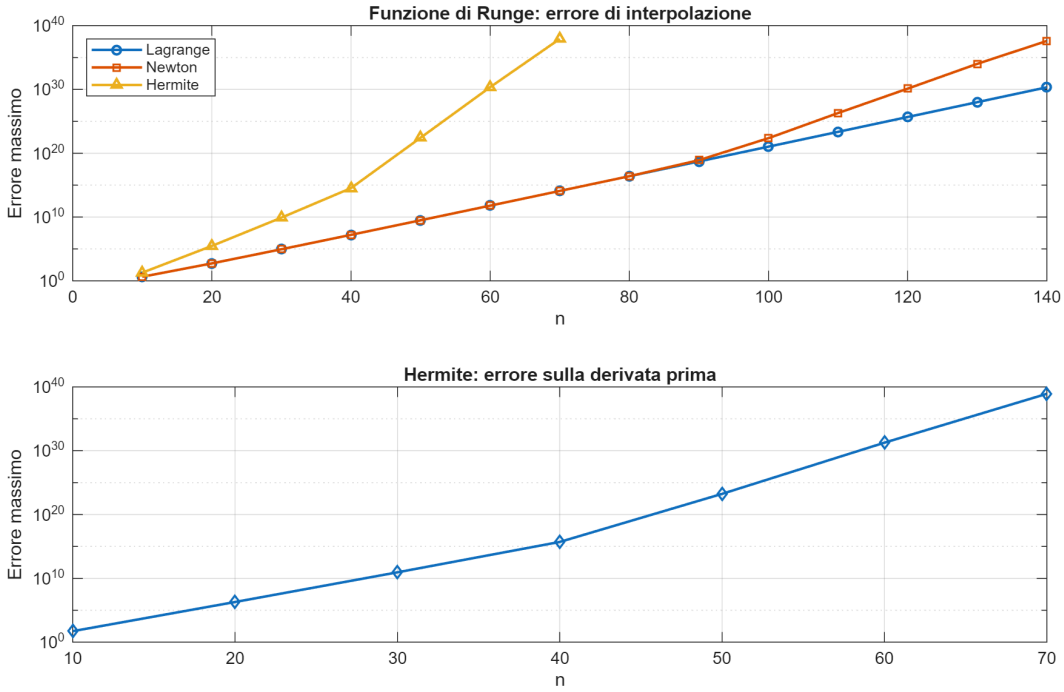



Figura 3: Errore massimo per l'interpolazione della funzione di Runge con ascisse equidistanti.

### Commento dei risultati

Il grafico mostra il fenomeno di Runge: con ascisse equidistanti l'errore aumenta al crescere del grado, soprattutto vicino agli estremi. Lagrange e Newton rappresentano lo stesso polinomio in aritmetica esatta; le curve si separano per gradi alti, quando gli arrotondamenti sono amplificati in modo diverso dalle due valutazioni.

L'errore passa da circa 4.34 per $n = 10$ a valori tra $10^{30}$ e $10^{37}$ per $n = 140$. Anche Hermite peggiora: imponendo pure le derivate si ottiene un polinomio di grado più alto, e per $n = 70$ gli errori su funzione e derivata sono rispettivamente circa $8.24 \cdot 10^{37}$ e $8.31 \cdot 10^{38}$. Le ascisse equidistanti sono quindi inadatte a questo esempio per gradi elevati.

↳ Per riprodurre i valori numerici e il grafico riportato, si faccia riferimento allo script MATLAB allegato denominato `test_es20.m`.

## Esercizio 21

Ripetere l'esercizio precedente, utilizzando le ascisse di Chebyshev in luogo di quelle equidistanti. Commentare i risultati ottenuti.

### Svolgimento

Si ripete il calcolo dell'esercizio 20 sostituendo le ascisse equidistanti con le ascisse di Chebyshev dell'esercizio 19:

$$x_i = \frac{a+b}{2} + \frac{b-a}{2}\cos\left((2i+1)\frac{\pi}{2(n+1)}\right), \quad i = 0, ..., n,$$

con $a = -10$ e $b = 10$. L'errore massimo è ancora stimato sui 10001 punti equispaziati dell'intervallo $[-10, 10]$, quindi si usano le stesse quantità

$$E_n = \max_{0 \le j \le 10000} |f(t_j) - p_{n(t_j)}|, \quad E'_n = \max_{0 \le j \le 10000} |f'(t_j) - p'_{n(t_j)}|$$

dell'esercizio precedente.

I nodi di Chebyshev minimizzano il massimo del prodotto nodale e portano a una crescita logaritmica della costante di Lebesgue; per questo sono più adatti dei nodi equidistanti quando il grado cresce.

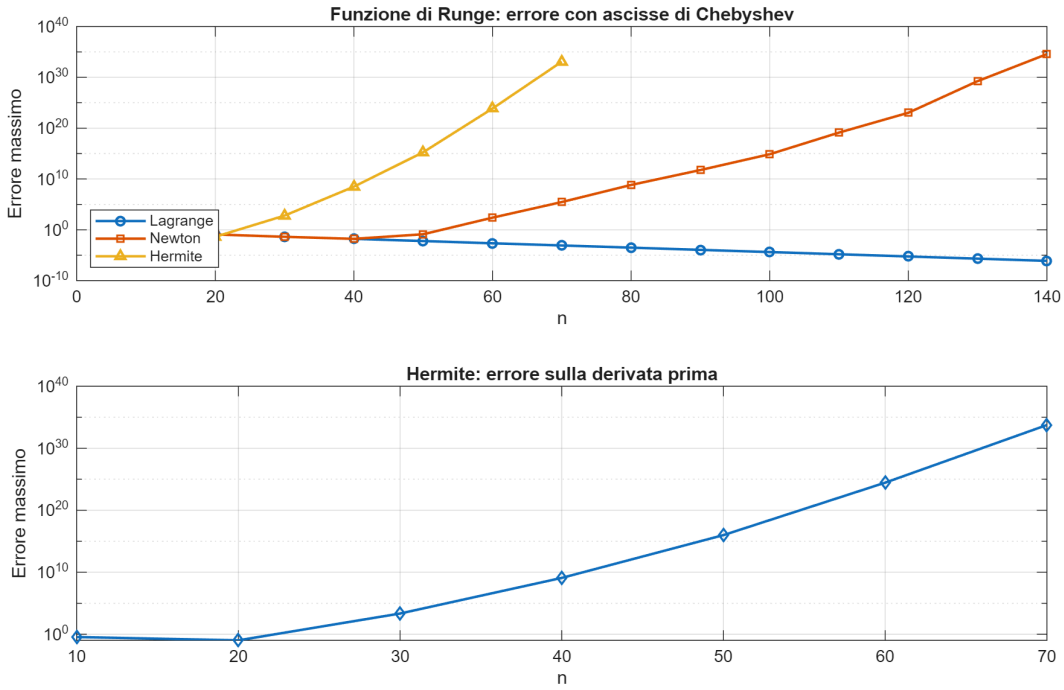



Figura 4: Errore massimo per l'interpolazione della funzione di Runge con ascisse di Chebyshev.

### Commento dei risultati

Il miglioramento rispetto ai nodi equidistanti è netto: per Lagrange l'errore scende da circa $3.59 \cdot 10^{-1}$ per $n = 10$ a $7.70 \cdot 10^{-7}$ per $n = 140$, senza il fenomeno di Runge osservato nell'esercizio 20.

La curva di Newton coincide con quella di Lagrange solo per gradi bassi; da circa $n = 50$ dominano gli errori di arrotondamento della forma di Newton. Per Hermite l'effetto è ancora più evidente, perché il grado è maggiore e le differenze divise usano nodi ripetuti. Le ascisse di Chebyshev migliorano quindi il problema di interpolazione, ma la stabilità della forma numerica resta decisiva.

↳ Per riprodurre i valori numerici e il grafico riportato, si faccia riferimento allo script MATLAB allegato denominato `test_es21.m` .

## Esercizio 22

Scrivere una function Matlab,

```
x = tridia( b, a, c, y )
```

che risolve un sistema tridiagonale, in cui `b` , `a` , `c` sono i vettori contenenti gli elementi sulle 3 diagonali, e `y` è un vettore contenente il termine noto.

### Svolgimento

Si considera il sistema tridiagonale nella forma:

$$b_i x_{i-1} + a_i x_i + c_i x_{i+1} = y_i, \quad i = 1, ..., n,$$

con $b_1 = 0$ e $c_n = 0$. La risoluzione applica l'eliminazione gaussiana solo sulle tre diagonali, cioè una fattorizzazione $A = LU$ implicita. Così non si costruisce mai la matrice completa e il costo resta lineare in $n$.

```
function x = tridia(b, a, c, y)
% Function x = tridia(b, a, c, y)
%
% Risolve un sistema lineare tridiagonale Ax = y di ordine n, dove
% b, a, c contengono rispettivamente sottodiagonale, diagonale
% principale e sopradiagonale:
%
%   b(i)*x(i-1) + a(i)*x(i) + c(i)*x(i+1) = y(i),
%   i = 1, ..., n, con b(1) = 0 e c(n) = 0.
%
% Input:
%   b - vettore della sottodiagonale;
%   a - vettore della diagonale principale;
%   c - vettore della sopradiagonale;
%   y - vettore dei termini noti.
%
% Output:
%   x - vettore soluzione del sistema.
%
% Rel. 17-05-2026.
```

```
% controlli di consistenza
if nargin < 4
    error('Numero di argomenti insufficiente.');
end

n = length(a);

if length(b) ~= n || length(c) ~= n || length(y) ~= n
    error('Dimensioni incompatibili tra b, a, c e y.');
end

if n == 0
    error('Il sistema non puo'' essere vuoto.');
end

% normalizzazione a vettori colonna
b = b(:);
a = a(:);
c = c(:);
x = y(:);

errore = 'Matrice non fattorizzabile senza pivoting.';

if a(1) == 0
    error(errore);
end

% eliminazione in avanti: A = L*U, memorizzando i moltiplicatori in b
for i = 2:n
    b(i) = b(i) / a(i - 1);
    a(i) = a(i) - b(i) * c(i - 1);

    if a(i) == 0
        error(errore);
    end

    x(i) = x(i) - b(i) * x(i - 1);
end

% sostituzione all'indietro sul sistema triangolare superiore
x(n) = x(n) / a(n);

for i = n-1:-1:1
    x(i) = (x(i) - c(i) * x(i + 1)) / a(i);
end

end
```

L'algoritmo usa solo vettori di lunghezza $n$ e richiede $O(n)$ operazioni. Il controllo sui pivot nulli serve a intercettare i casi in cui la fattorizzazione senza pivoting non è possibile; in particolare, per matrici tridiagonali a diagonale dominante la fattorizzazione esiste.

↳ Il file MATLAB allegato è `tridia.m` .

## Esercizio 23

Dimostrare che, per una spline cubica completa interpolante una funzione $f(x)$ sulla partizione

$$\Delta = \{a = x_0 < x_1 < ... < x_n = b\}, \quad h_i = x_i - x_{i-1}, \quad i = 1, ..., n,$$

i valori

$$m_i = s_3''(x_i), \quad i = 0, ..., n,$$

sono la soluzione del sistema lineare tridiagonale con matrice a diagonale dominante per righe:

$$\begin{pmatrix} 2 & 1 & 0 & ... & 0 & 0 \\ \varphi_1 & 2 & \xi_1 & ... & 0 & 0 \\ 0 & \varphi_2 & 2 & \xi_2 & ... & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & ... & \varphi_{n-1} & 2 & \xi_{n-1} \\ 0 & 0 & ... & 0 & 1 & 2 \end{pmatrix} \begin{pmatrix} m_0 \\ m_1 \\ m_2 \\ \vdots \\ m_{n-1} \\ m_n \end{pmatrix} = 6 \begin{pmatrix} f[x_0, x_0, x_1] \\ f[x_0, x_1, x_2] \\ f[x_1, x_2, x_3] \\ \vdots \\ f[x_{n-2}, x_{n-1}, x_n] \\ f[x_{n-1}, x_n, x_n] \end{pmatrix},$$

con

$$\varphi_i = \frac{h_i}{h_i + h_{i+1}}, \quad \xi_i = \frac{h_{i+1}}{h_i + h_{i+1}}, \quad i = 1, ..., n-1.$$

### Svolgimento

Poniamo $f_i = f(x_i)$ e indichiamo con $s_i$ la restrizione della spline all'intervallo $[x_{i-1}, x_i]$. Usiamo la convenzione delle differenze divise con nodi ripetuti, quindi $f[x_0, x_0] = f'(x_0)$ e $f[x_n, x_n] = f'(x_n)$. Poiche' $s_{i''}$ e' lineare su ciascun intervallo e vale $m_{i-1}$ e $m_i$ agli estremi, si ha

$$s_{i''}(x) = m_{i-1}\frac{x_i - x}{h_i} + m_i\frac{x - x_{i-1}}{h_i}, \quad x \in [x_{i-1}, x_i].$$

Integrando due volte e imponendo $s_{i(x_{i-1})} = f_{i-1}$, $s_{i(x_i)} = f_i$, si ottiene la formula locale

$$s_{i(x)} = m_{i-1}\frac{(x_i - x)^3}{6h_i} + m_i\frac{(x - x_{i-1})^3}{6h_i} + \left(\frac{f_{i-1}}{h_i} - m_{i-1}\frac{h_i}{6}\right)(x_i - x) + \left(\frac{f_i}{h_i} - m_i\frac{h_i}{6}\right)(x - x_{i-1}).$$

Derivando e valutando agli estremi dell'intervallo segue

$$s_{i'}(x_{i-1}) = f[x_{i-1}, x_i] - \frac{h_i}{6}(2m_{i-1} + m_i), \quad s_{i'}(x_i) = f[x_{i-1}, x_i] + \frac{h_i}{6}(m_{i-1} + 2m_i).$$

Nei nodi interni $x_i$, $i = 1, ..., n-1$, la spline deve avere derivata prima continua, quindi $s_{i'}(x_i) = s_{(i+1)'}(x_i)$. Usando le due formule precedenti:

$$f[x_{i-1}, x_i] + \frac{h_i}{6}(m_{i-1} + 2m_i) = f[x_i, x_{i+1}] - \frac{h_{i+1}}{6}(2m_i + m_{i+1}).$$

Portando i termini con $m$ a sinistra:

$$h_i m_{i-1} + 2(h_i + h_{i+1})m_i + h_{i+1}m_{i+1} = 6(f[x_i, x_{i+1}] - f[x_{i-1}, x_i]).$$

Dividendo per $h_i + h_{i+1}$ si ricava

$$\varphi_i m_{i-1} + 2m_i + \xi_i m_{i+1} = 6f[x_{i-1}, x_i, x_{i+1}], \quad i = 1, ..., n-1,$$

perché

$$f[x_{i-1}, x_i, x_{i+1}] = \frac{f[x_i, x_{i+1}] - f[x_{i-1}, x_i]}{h_i + h_{i+1}}.$$

Restano le due condizioni che rendono la spline completa:

$$s_{1'}(x_0) = f'(x_0) = f[x_0, x_0], \quad s_{n'}(x_n) = f'(x_n) = f[x_n, x_n].$$

La prima, applicata alla formula di $s_{1'}(x_0)$, dà

$$f[x_0, x_1] - \frac{h_1}{6}(2m_0 + m_1) = f[x_0, x_0],$$

cioè

$$2m_0 + m_1 = 6\frac{f[x_0, x_1] - f[x_0, x_0]}{h_1} = 6f[x_0, x_0, x_1].$$

Analogamente, dalla condizione destra,

$$f[x_{n-1}, x_n] + \frac{h_n}{6}(m_{n-1} + 2m_n) = f[x_n, x_n],$$

quindi

$$m_{n-1} + 2m_n = 6\frac{f[x_n, x_n] - f[x_{n-1}, x_n]}{h_n} = 6f[x_{n-1}, x_n, x_n].$$

Le due equazioni di bordo e le $n-1$ equazioni interne danno esattamente il sistema indicato nel testo. Infine, poiché $h_i > 0$, valgono $\varphi_i > 0$, $\xi_i > 0$ e $\varphi_i + \xi_i = 1$. Dunque nelle righe interne il coefficiente diagonale vale 2 e la somma dei coefficienti fuori diagonale vale 1; nelle righe estreme vale ancora $2 > 1$. La matrice è quindi a diagonale dominante stretta per righe, e in particolare è non singolare. Il sistema ammette quindi una sola soluzione, che coincide con il vettore delle derivate seconde nodali della spline completa.

## Esercizio 24

Utilizzando la function del precedente esercizio 22, scrivere la function Matlab,

```
y = spline3( xi, fi, x, type, f1a, f1b )
```

in cui `(xi, fi)` sono i dati di interpolazione e `x` è un vettore (riga o colonna), contenente i punti in cui valutare la spline cubica interpolante, restituiti nel vettore `y`, che avrà le stesse dimensioni di `x`. Infine, `type` è un flag che vale 1 (valore di default) per la spline cubica not-a-knot, 0 per quella naturale, 2 per la spline cubica completa. In quest'ultimo caso, gli ultimi due parametri di ingresso (opzionali) contengono il valore della derivata della funzione interpolante negli estremi.

## Svolgimento

Si cercano prima i valori delle derivate seconde nodali

$$m_i = s''(\xi_i)$$

nei nodi. Una volta noti questi valori, su ciascun intervallo $[\xi_i, \xi_{i+1}]$ la spline si valuta con la formula

$$s(x) = m_i \frac{(\xi_{i+1} - x)^3}{6h_i} + m_{i+1} \frac{(x - \xi_i)^3}{6h_i} + \left( \frac{f_i}{h_i} - m_i \frac{h_i}{6} \right) (\xi_{i+1} - x) + \left( \frac{f_{i+1}}{h_i} - m_{i+1} \frac{h_i}{6} \right) (x - \xi_i),$$

dove $h_i = \xi_{i+1} - \xi_i$.

Le equazioni interne:

$$\varphi_i m_{i-1} + 2m_i + \eta_i m_{i+1} = 6 \frac{f[x_i, x_{i+1}] - f[x_{i-1}, x_i]}{h_{i-1} + h_i},$$

con

$$\varphi_i = \frac{h_{i-1}}{h_{i-1} + h_i}, \quad \eta_i = \frac{h_i}{h_{i-1} + h_i}.$$

Le condizioni di bordo dipendono da `type`: per la spline naturale si impone $m_0 = m_n = 0$, per quella completa si usano le derivate assegnate agli estremi, mentre per la not-a-knot si impone la continuità della derivata terza nel primo e nell'ultimo nodo interno.

```
function y = spline3(xi, fi, x, type, f1a, f1b)
% Function y = spline3(xi, fi, x, type, f1a, f1b)
%
% Valuta la spline cubica interpolante associata ai dati (xi, fi)
% nei punti contenuti nel vettore x.
%
% Input:
%   xi   - vettore delle ascisse di interpolazione;
%   fi   - vettore dei valori della funzione nelle ascisse xi;
```

```
%   x    - vettore dei punti di valutazione;
%   type - tipo di spline:
%          0 naturale, 1 not-a-knot, 2 completa.
%   f1a  - derivata prima nell'estremo sinistro, per type = 2;
%   f1b  - derivata prima nell'estremo destro, per type = 2.
%
% Output:
%   y    - valori della spline nei punti x, con le stesse dimensioni di x.
%
% Rel. 18-05-2026.

% controlli di consistenza
if nargin < 3
    error('Numero di argomenti insufficiente.');
end

if nargin < 4 || isempty(type)
    type = 1;
end

if ~isscalar(type) || ~ismember(type, [0 1 2])
    error('Il parametro type deve valere 0, 1 oppure 2.');
end

n = length(xi);

if length(fi) ~= n
    error('Dimensioni incompatibili tra xi e fi.');
end

if n < 2
    error('Servono almeno due ascisse di interpolazione.');
end

if length(unique(xi)) ~= n
    error('Le ascisse di interpolazione devono essere distinte.');
end

% uso vettori colonna per trattare allo stesso modo input riga o colonna
xi = xi(:);
fi = fi(:);

h = diff(xi);

if any(h <= 0)
    error('Le ascisse xi devono essere strettamente crescenti.');
end
```

```
d = diff(fi) ./ h;

% calcolo delle derivate seconde nodali m(i) = s''(xi(i))
switch type
    case 0
        m = spline_naturale(h, d, n);

    case 1
        if n < 4
            error('La spline not-a-knot richiede almeno quattro ascisse.');
        end

        m = spline_notaknot(h, d, n);

    case 2
        if nargin < 6
            error('Per la spline completa servono f1a e f1b.');
        end

        m = spline_completa(h, d, n, f1a, f1b);
end

% lavoro sui punti di valutazione come vettore colonna
xx = x(:);

% ind(k) indica il tratto [xi(ind(k)), xi(ind(k)+1)] che contiene xx(k)
[~, ind] = histc(xx, xi);

% xx < xi(1) e xx >= xi(n) sono vettori logici: selezionano in ind
% solo le posizioni dei punti esterni, da valutare col primo o ultimo tratto
ind(xx < xi(1)) = 1;
ind(xx >= xi(n)) = n - 1;
ind(ind == 0) = 1;

hi = h(ind);
sinistra = xi(ind + 1) - xx;
destra = xx - xi(ind);

% valutazione della formula locale della spline cubica
yy = m(ind) .* sinistra .^ 3 ./ (6 * hi) ...
    + m(ind + 1) .* destra .^ 3 ./ (6 * hi) ...
    + (fi(ind) ./ hi - m(ind) .* hi / 6) .* sinistra ...
    + (fi(ind + 1) ./ hi - m(ind + 1) .* hi / 6) .* destra;

% ripristino la stessa forma di x: riga se x era riga, colonna se era colonna
y = reshape(yy, size(x));
```

```
end

function m = spline_naturale(h, d, n)

b = zeros(n, 1);
a = zeros(n, 1);
c = zeros(n, 1);
rhs = zeros(n, 1);

a(1) = 1;
a(n) = 1;

for i = 2:n-1
    den = h(i - 1) + h(i);
    b(i) = h(i - 1) / den;
    a(i) = 2;
    c(i) = h(i) / den;
    rhs(i) = 6 * (d(i) - d(i - 1)) / den;
end

m = tridia(b, a, c, rhs);

end

function m = spline_completa(h, d, n, f1a, f1b)

b = zeros(n, 1);
a = zeros(n, 1);
c = zeros(n, 1);
rhs = zeros(n, 1);

a(1) = 2;
c(1) = 1;
rhs(1) = 6 * (d(1) - f1a) / h(1);

for i = 2:n-1
    den = h(i - 1) + h(i);
    b(i) = h(i - 1) / den;
    a(i) = 2;
    c(i) = h(i) / den;
    rhs(i) = 6 * (d(i) - d(i - 1)) / den;
end

b(n) = 1;
a(n) = 2;
rhs(n) = 6 * (f1b - d(n - 1)) / h(n - 1);
```

```
m = tridia(b, a, c, rhs);

end

function m = spline_notaknot(h, d, n)

N = n - 2;
b = zeros(N, 1);
a = zeros(N, 1);
c = zeros(N, 1);
rhs = zeros(N, 1);

for j = 1:N
    i = j + 1;
    den = h(i - 1) + h(i);
    rhs(j) = 6 * (d(i) - d(i - 1)) / den;

    if j == 1
        a(j) = 2 + h(1) / h(2);
        c(j) = (h(2) - h(1)) / h(2);
    elseif j == N
        b(j) = (h(n - 2) - h(n - 1)) / h(n - 2);
        a(j) = 2 + h(n - 1) / h(n - 2);
    else
        b(j) = h(i - 1) / den;
        a(j) = 2;
        c(j) = h(i) / den;
    end
end

u = tridia(b, a, c, rhs);

m = zeros(n, 1);
m(2:n-1) = u;
m(1) = (h(1) + h(2)) / h(2) * m(2) - h(1) / h(2) * m(3);
m(n) = (h(n - 2) + h(n - 1)) / h(n - 2) * m(n - 1) ...
    - h(n - 1) / h(n - 2) * m(n - 2);

end
```

La function costruisce solo il sistema tridiagonale delle derivate seconde nodali e lo risolve con `tridia`. La valutazione usa poi la formula locale della spline e preserva la forma del vettore `x`, anche quando è un vettore riga.

# Esercizio 25

Utilizzare la function `spline3` per approssimare la funzione di Runge, come nell'esercizio 20, utilizzando $n+1$ ascisse equidistanti, con $n = 100, 200, ..., 2000$. Detta $h = \frac{20}{n}$ la distanza tra due punti consecutivi, graficare, in formato loglog l'errore massimo, calcolato come nell'esercizio 20, rispetto ad $h$. Con che potenza di $h$ decresce, asintoticamente l'errore, per le tre spline interpolanti?

## Svolgimento

Si usa lo stesso criterio dell'esercizio 20. Fissati i 10001 punti equispaziati

$$t_j = -10 + j \cdot \frac{20}{10000}, \quad j = 0, ..., 10000,$$

l'errore massimo viene calcolato come

$$E_n = \max_{0 \le j \le 10000} |f(t_j) - s_{n(t_j)}|, \quad f(x) = \frac{1}{1+x^2}.$$

Per la spline completa sono state usate le derivate esatte agli estremi:

$$f'(x) = -\frac{2x}{(1+x^2)^2}.$$

Nelle chiamate a `spline3`, `type = 0` corrisponde alla spline naturale, `type = 1` alla not-a-knot e `type = 2` alla completa.

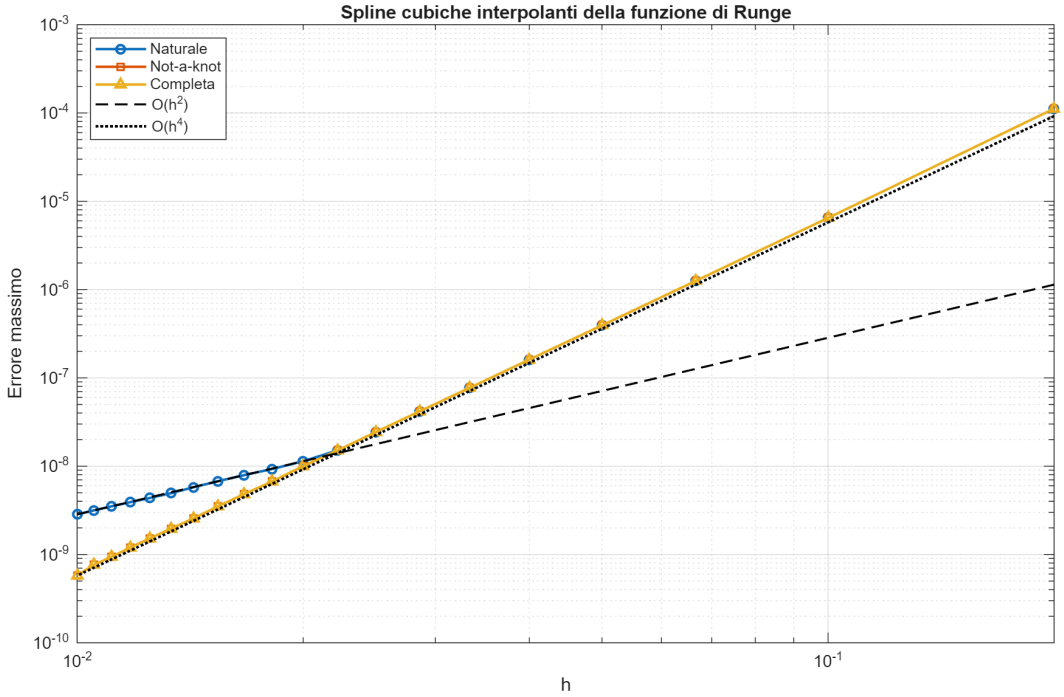



Figura 5: Errore massimo delle tre spline cubiche interpolanti per la funzione di Runge.

Per stimare la potenza di $h$ si è approssimata con una retta la parte finale del grafico nel piano $\log(E_n)$-$\log(h)$, usando gli ultimi dieci valori di $h$. Se $E_n \approx Ch^p$, allora la pendenza della retta fornisce proprio una stima di $p$.

*Ordini asintotici stimati dal grafico loglog, usando gli ultimi dieci valori di $h$.*

| Spline | Ordine stimato | $E_{1000}$ | $E_{2000}$ |
|---|---|---|---|
| Naturale | 1.9908 | $1.1375 \cdot 10^{-8}$ | $2.8437 \cdot 10^{-9}$ |
| Not-a-knot | 4.0475 | $1.0015 \cdot 10^{-8}$ | $5.7631 \cdot 10^{-10}$ |
| Completa | 4.0475 | $1.0015 \cdot 10^{-8}$ | $5.7631 \cdot 10^{-10}$ |

**Commento**

Asintoticamente la spline naturale decresce come $h^2$, mentre not-a-knot e completa come $h^4$. La differenza dipende dalle condizioni al bordo: la completa usa le derivate prime corrette e la not-a-knot mantiene il quarto ordine; la naturale impone invece $s''(-10) = s''(10) = 0$, ma per la funzione di Runge

$$f''(x) = \frac{6x^2 - 2}{(1 + x^2)^3}$$

e quindi $f''(\pm 10) \neq 0$. Il vincolo naturale non è compatibile con la funzione interpolata e l'errore globale resta di ordine $h^2$.

Per griglie non ancora fini le curve sono quasi sovrapposte; il diverso ordine della spline naturale emerge solo nel regime asintotico.

↳ Per riprodurre i valori numerici e il grafico riportato, si faccia riferimento allo script MATLAB allegato denominato `test_es25.m` .

## Esercizio 26

> Ripetere il precedente esercizio considerando, invece dell'intervallo $[-10, 10]$, l'intervallo $[-5, 15]$. Commentare i risultati ottenuti, rispetto a quelli del precedente esercizio.

### Svolgimento

Si mantiene lo stesso procedimento dell'esercizio 25, cambiando solo l'intervallo di interpolazione. I punti di controllo dell'errore sono quindi

$$t_j = -5 + j \cdot \frac{20}{10000}, \quad j = 0, ..., 10000,$$

e, per ogni $n = 100, 200, ..., 2000$, si pone ancora $h = \frac{20}{n}$ e

$$E_n = \max_{0 \le j \le 10000} |f(t_j) - s_{n(t_j)}|, \quad f(x) = \frac{1}{1+x^2}.$$

Per la spline completa si usano le derivate esatte agli estremi:

$$f'(x) = -\frac{2x}{(1+x^2)^2}.$$

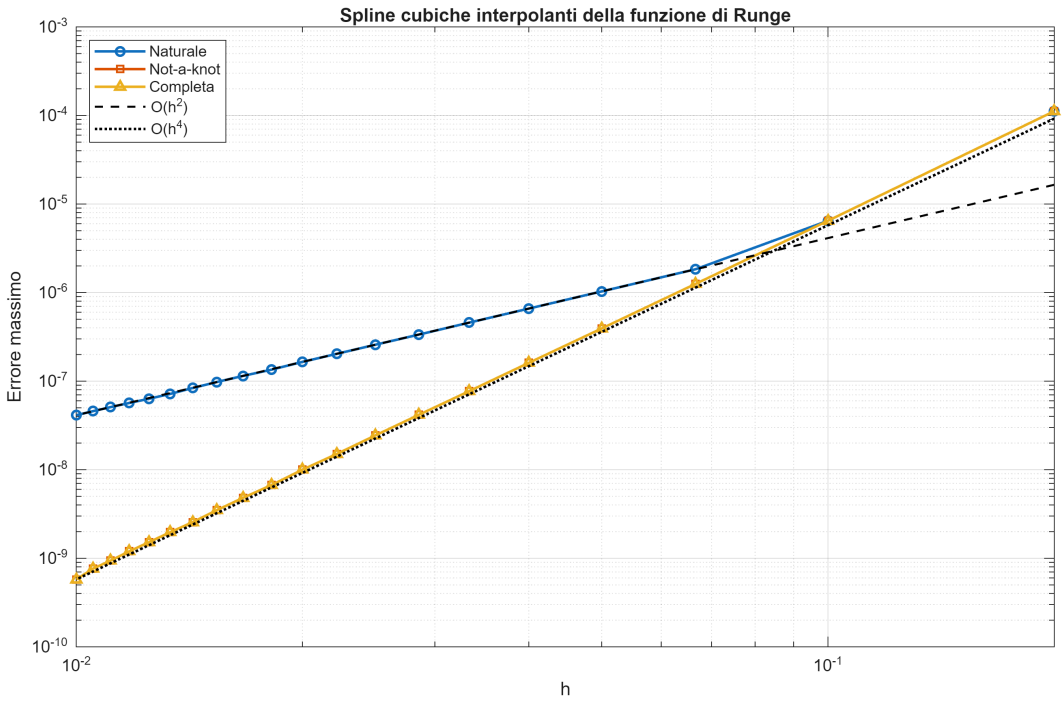



Figura 6: Errore massimo delle tre spline cubiche interpolanti sull'intervallo $[-5, 15]$.

Come nell'esercizio precedente, la potenza di $h$ è stata stimata approssimando con una retta la parte finale del grafico nel piano $\log(E_n)$-$\log(h)$, usando gli ultimi dieci valori di $h$.

*Ordini asintotici stimati dal grafico loglog, usando gli ultimi dieci valori di $h$.*

| Spline | Ordine stimato | $E_{1000}$ | $E_{2000}$ |
|---|---|---|---|
| Naturale | 1.9908 | $1.6502 \cdot 10^{-7}$ | $4.1256 \cdot 10^{-8}$ |
| Not-a-knot | 4.0475 | $1.0015 \cdot 10^{-8}$ | $5.7631 \cdot 10^{-10}$ |
| Completa | 4.0475 | $1.0015 \cdot 10^{-8}$ | $5.7631 \cdot 10^{-10}$ |

### Commento

Gli ordini asintotici restano quelli dell'esercizio 25: $h^2$ per la spline naturale e $h^4$ per not-a-knot e completa. Cambia invece la costante dell'errore della spline naturale. Sul nuovo intervallo il vincolo naturale impone $s''(-5) = s''(15) = 0$, ma

$$f''(x) = \frac{6x^2 - 2}{(1+x^2)^3}$$

e in particolare $f''(-5) \approx 8.42 \cdot 10^{-3}$, molto più grande di $f''(\pm 10) \approx 5.80 \cdot 10^{-4}$ dell'esercizio precedente. Per questo l'errore della spline naturale è circa quindici volte più grande: ad esempio $E_{2000}$ passa da $2.8437 \cdot 10^{-9}$ a $4.1256 \cdot 10^{-8}$.

Le curve not-a-knot e completa restano praticamente coincidenti con quelle dell'esercizio 25: il massimo dell'errore è vicino a $x = 0$ e il passo $h$ è lo stesso nei due intervalli.

↳ Per riprodurre i valori numerici e il grafico riportato, si faccia riferimento allo script MATLAB allegato denominato `test_es26.m`.

## Esercizio 27

Costruire una function Matlab,

```
ci = ncotes( n )
```

che calcoli i coefficienti della formula di Newton-Cotes di grado $n$. Costruire una tabella con tutti i coefficienti delle formule ammissibili per ragioni di condizionamento. I coefficienti della tabella devono essere riportati come numeri razionali.

### Svolgimento

Per la formula chiusa di Newton-Cotes si prendono i nodi $x_i = a + ih$, con $i = 0, ..., n$ e $h = \frac{b-a}{n}$. Scrivendo $x = a + ht$, i coefficienti sono

$$c_i = \int_0^n L_{i(t)}\,\mathrm{d}t, \quad L_{i(t)} = \prod_{j=0, j\neq i}^{n} \frac{t-j}{i-j},$$

e la formula assume la forma

$$I_{n(f)} = h \sum_{i=0}^{n} c_i f(x_i).$$

Quindi è sufficiente costruire il polinomio fondamentale di Lagrange relativo al nodo $i$ e integrarlo esattamente.

```
function ci = ncotes(n)
% Function ci = ncotes(n)
%
% Calcola i coefficienti della formula chiusa di Newton-Cotes
% di grado n sui nodi equidistanti 0, 1, ..., n.
%
% La formula risultante e'
%
%   I_n(f) = h * sum_{i=0}^n ci(i+1) * f(x_i),
%
% dove x_i = a + i*h e h = (b-a)/n.
%
```

```
% Input:
%   n  - grado della formula di Newton-Cotes.
%
% Output:
%   ci - vettore colonna dei coefficienti della formula.
%
% Rel. 20-05-2026.

% controlli di consistenza
if nargin < 1
    error('Numero di argomenti insufficiente.');
end

if ~isscalar(n) || n ~= fix(n) || n < 1
    error('Il grado n deve essere un intero positivo.');
end

ci = zeros(n + 1, 1);

% Il coefficiente ci(i+1) e' l'integrale su [0,n] del polinomio
% fondamentale di Lagrange relativo al nodo i.
for i = 0:n
    ind = [0:i-1 i+1:n];

    den = prod(i - ind);
    p = poly(ind);

    pint = [p ./ (n + 1:-1:1), 0];
    ci(i + 1) = polyval(pint, n) / den;
end

end
```

Il condizionamento della formula discreta è governato da $h\sum_i |c_i|$. Poiché $\sum_i c_i = n$, se tutti i coefficienti sono non negativi il numero di condizionamento discreto coincide con quello del problema continuo; altrimenti peggiora. Come indicato nelle dispense, i gradi ammissibili sono dunque $n = 1, ..., 7$ e $n = 9$.

*Coefficienti $c_i$ tali che $I_{n(f)} = h\sum_i c_i f(x_i)$.*

| $n$ | $c_0, ..., c_n$ |
|---|---|
| 1 | $\frac{1}{2}(1, 1)$ |
| 2 | $\frac{1}{3}(1, 4, 1)$ |
| 3 | $\frac{3}{8}(1, 3, 3, 1)$ |
| 4 | $\frac{1}{45}(14, 64, 24, 64, 14)$ |
| 5 | $\frac{1}{288}(95, 375, 250, 250, 375, 95)$ |

| | |
|---|---|
| 6 | $\frac{1}{140}(41, 216, 27, 272, 27, 216, 41)$ |
| 7 | $\frac{1}{17280}(5257, 25039, 9261, 20923, 20923, 9261, 25039, 5257)$ |
| 9 | $\frac{1}{89600}(25713, 141669, 9720, 174096, 52002, 52002, 174096, 9720, 141669, 25713)$ |

Il grado 8 non compare nella tabella perché produce coefficienti negativi e quindi una formula peggio condizionata rispetto all'integrale continuo.

## Esercizio 28

Costruire una function Matlab,

```
[Ikn,err] = ncotescomp( fx, a, b, k, n )
```

che calcoli l'approssimazione dell'integrale della funzione `fx` sull'intervallo $[a, b]$, utilizzando la formula di Newton-Cotes di grado $k$ su $n + 1$ ascisse equidistanti, con $n$ multiplo pari di $k$, insieme ad una stima dell'errore di quadratura.

### Svolgimento

Siano

$$x_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, ..., n.$$

Se $n = k\ell$, la formula composita si ottiene applicando la formula chiusa di Newton-Cotes di grado $k$ su ciascun blocco $\left[x_{jk}, x_{(j+1)k}\right]$:

$$I_n^{k(f)} = h \sum_{j=0}^{\ell-1} \sum_{i=0}^{k} c_i f\left(x_{jk+i}\right).$$

I coefficienti $c_i$ sono quelli calcolati dalla function `ncotes` dell'esercizio precedente.

La richiesta che $n$ sia un multiplo pari di $k$ serve per costruire, senza nuove valutazioni di funzione, anche la formula composita sui soli nodi di indice pari, cioè con passo $2h$. Indicando con $p$ l'ordine dell'errore della formula composita,

$$p = \begin{cases} k+1 \\ \text{se } \ k \ \text{ e' dispari} \\ k+2 \\ \text{se } \ k \ \text{ e' pari} \end{cases},$$

si usa la stima

$$\text{err} \approx |I_n^{k(f)} - I_{\frac{n}{2}}^{k(f)}| \frac{}{2^p - 1}.$$

Come nelle dispense e nell'esercizio 27, il codice accetta solo i gradi ammissibili per condizionamento: $k = 1, ..., 7$ e $k = 9$.

```
function [Ikn, err] = ncotescomp(fx, a, b, k, n)
% Function [Ikn, err] = ncotescomp(fx, a, b, k, n)
%
% Calcola la formula composita di Newton-Cotes chiusa di grado k
% sull'intervallo [a,b], usando n + 1 ascisse equidistanti.
% Il parametro n deve essere un multiplo pari di k, cosi' da poter
% costruire anche la formula sui soli nodi di indice pari.
%
% Input:
%   fx - function handle della funzione integranda;
%   a  - estremo sinistro dell'intervallo;
%   b  - estremo destro dell'intervallo;
%   k  - grado della formula elementare di Newton-Cotes;
%   n  - numero di sottointervalli.
%
% Output:
%   Ikn - approssimazione dell'integrale;
%   err - stima dell'errore di quadratura.
%
% Rel. 20-05-2026.

% controlli di consistenza
if nargin < 5
    error('Numero di argomenti insufficiente.');
end

if a >= b
    error('L''intervallo [a, b] non e'' valido.');
end

if ~isscalar(k) || k ~= fix(k) || ~ismember(k, [1:7 9])
    error('Il grado k deve appartenere all''insieme [1:7 9].');
end

if ~isscalar(n) || n ~= fix(n) || n < k || ...
        mod(n, k) ~= 0 || mod(n / k, 2) ~= 0
    error('n deve essere un multiplo pari di k.');
end

ci = ncotes(k);
h = (b - a) / n;
x = linspace(a, b, n + 1);
fi = zeros(n + 1, 1);
```

```
for i = 1:n+1
    fi(i) = fx(x(i));
end

Ikn = formula_composita(fi, ci, h, k, n, 1);
Icoarse = formula_composita(fi, ci, 2 * h, k, n, 2);

if mod(k, 2) == 0
    p = k + 2;
else
    p = k + 1;
end

err = abs(Ikn - Icoarse) / (2 ^ p - 1);

end

function I = formula_composita(fi, ci, h, k, n, passo)

num_blocchi = n / (k * passo);
I = 0;

for j = 0:num_blocchi-1
    ind = j * k * passo + (0:k) * passo + 1;
    I = I + h * (ci.' * fi(ind));
end

end
```

## Esercizio 29

> Scrivere una function Matlab che calcoli l'approssimazione di un integrale definito mediante la formula adattiva di Simpson con una tolleranza prescritta `tol` .

### Svolgimento

Sia $m = \frac{a+b}{2}$. La formula di Simpson sull'intervallo $[a, b]$ è

$$S_1 = \frac{b-a}{6}(f(a) + 4f(m) + f(b)).$$

Per decidere se l'approssimazione è sufficiente, si confronta $S_1$ con la formula ottenuta applicando Simpson sui due sottointervalli:

$$S_2 = S(a,m) + S(m,b), \quad \text{err} = \frac{|S_2 - S_1|}{15}.$$

Se err $\leq$ tol si accetta $S_2$; altrimenti si ripete lo stesso procedimento su $[a,m]$ e $[m,b]$, assegnando tolleranza `"tol" / 2` a ciascun sottointervallo. I valori di funzione già calcolati vengono passati alla ricorsione per evitare valutazioni ridondanti.

```
function [I, err, nfev] = simpsonad(fx, a, b, tol)
% Function [I, err, nfev] = simpsonad(fx, a, b, tol)
%
% Calcola l'integrale di fx in [a,b] mediante la formula adattiva
% di Simpson con tolleranza tol.
%
% Input:
%   fx  - function handle della funzione integranda;
%   a   - estremo sinistro dell'intervallo;
%   b   - estremo destro dell'intervallo;
%   tol - tolleranza richiesta.
%
% Output:
%   I    - approssimazione dell'integrale;
%   err  - stima totale dell'errore;
%   nfev - numero di valutazioni funzionali effettuate.
%
% Rel. 22-05-2026.

% controlli di consistenza
if nargin < 4
    error('Numero di argomenti insufficiente.');
end

if a >= b
    error('L''intervallo [a, b] non e'' valido.');
end

if tol <= 0
    error('La tolleranza deve essere positiva.');
end

m = (a + b) / 2;
fa = fx(a);
fm = fx(m);
fb = fx(b);

[I, err, nfev_rec] = simpsonad_rec(fx, a, b, tol, fa, fm, fb);
% aggiungo le tre valutazioni iniziali: f(a), f(m), f(b)
nfev = nfev_rec + 3;
```

```
end

function [I, err, nfev] = simpsonad_rec(fx, a, b, tol, fa, fm, fb)
% Funzione ricorsiva: riusa fa, fm e fb gia' calcolati.

m = (a + b) / 2;
x1 = (a + m) / 2;
x2 = (m + b) / 2;

f1 = fx(x1);
f2 = fx(x2);
nfev = 2;

S1 = simpson(a, b, fa, fm, fb);
Ssx = simpson(a, m, fa, f1, fm);
Sdx = simpson(m, b, fm, f2, fb);
S2 = Ssx + Sdx;

err = abs(S2 - S1) / 15;

if err <= tol
    I = S2;
else
    [Isx, err_sx, nfev_sx] = simpsonad_rec(fx, a, m, ...
        tol / 2, fa, f1, fm);
    [Idx, err_dx, nfev_dx] = simpsonad_rec(fx, m, b, ...
        tol / 2, fm, f2, fb);

    I = Isx + Idx;
    err = err_sx + err_dx;
    nfev = nfev + nfev_sx + nfev_dx;
end

end

function S = simpson(a, b, fa, fm, fb)
% Formula elementare di Simpson su [a,b].

S = (b - a) * (fa + 4 * fm + fb) / 6;

end
```

## Esercizio 30

Utilizzare le function dei precedenti due esercizi per approssimare

$$\int_0^1 10^2 \sin(10^2 x) e^{-10^2 x} \, \mathrm{d}x \approx \frac{1}{2},$$

con la formula composita, e quella adattativa, di Simpson. Per la formula composita, tabulare l'errore e la sua stima, per $n = 40, 80, 160, 320$. Per la formula adattiva, tabulare l'errore e `tol`, per tol $= 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}$. In entrambi i casi, riportare il numero di valutazioni funzionali effettuate.

## Svolgimento

Si prende come valore di riferimento

$$I = \int_0^1 100 \sin(100x) e^{-100x} \, \mathrm{d}x = \frac{1 - e^{-100}(\sin(100) + \cos(100))}{2}.$$

In doppia precisione questo valore coincide con $\frac{1}{2}$, perché il termine con $e^{-100}$ è dell'ordine di $10^{-44}$.

Per la formula composita di Simpson si usa `ncotescomp` con $k = 2$. Indico con $S_n$ il valore ottenuto usando $n+1$ nodi equidistanti. La stima dell'errore è ottenuta confrontando la formula su tutti i nodi con quella sui soli nodi di indice pari; in questo caso l'ordine è $p = 4$, quindi

$$\mathrm{err} = \frac{\left| S_n - S_{\frac{n}{2}} \right|}{2^4 - 1}.$$

Le valutazioni funzionali sono $n+1$, perché la formula con passo doppio riusa valori già calcolati.

*Formula composita di Simpson per* $f(x) = 100 \sin(100x) e^{-100x}$.

| $n$ | $S_n$ | Errore | Stima | Valutazioni |
|---|---|---|---|---|
| 40 | $1.5467 \cdot 10^{-1}$ | $3.4533 \cdot 10^{-1}$ | $1.3188 \cdot 10^{-2}$ | 41 |
| 80 | $4.6675 \cdot 10^{-1}$ | $3.3251 \cdot 10^{-2}$ | $2.0805 \cdot 10^{-2}$ | 81 |
| 160 | $4.9816 \cdot 10^{-1}$ | $1.8392 \cdot 10^{-3}$ | $2.0941 \cdot 10^{-3}$ | 161 |
| 320 | $4.9989 \cdot 10^{-1}$ | $1.0838 \cdot 10^{-4}$ | $1.1539 \cdot 10^{-4}$ | 321 |

Per la formula adattiva si applica direttamente `simpsonad` all'intervallo $[0, 1]$, con le tolleranze richieste.

*Formula adattiva di Simpson applicata su* $[0, 1]$.

| tol | $I$ | Errore | Valutazioni |
|---|---|---|---|
| $10^{-3}$ | $-6.1270 \cdot 10^{-11}$ | $5.0000 \cdot 10^{-1}$ | 5 |
| $10^{-4}$ | $-6.1270 \cdot 10^{-11}$ | $5.0000 \cdot 10^{-1}$ | 5 |

| | | | |
|---|---|---|---|
| $10^{-5}$ | $-6.1270 \cdot 10^{-11}$ | $5.0000 \cdot 10^{-1}$ | 5 |
| $10^{-6}$ | $-6.1270 \cdot 10^{-11}$ | $5.0000 \cdot 10^{-1}$ | 5 |

Il risultato adattivo non migliora al diminuire di `"tol"`. Al primo passo la routine confronta Simpson su $[0,1]$ con la somma delle formule su $\left[0, \frac{1}{2}\right]$ e $\left[\frac{1}{2}, 1\right]$. La stima locale usata dal test adattivo è

$$\frac{|S_2 - S_1|}{15} \approx 4.08 \cdot 10^{-12}.$$

Questo valore è già minore di tutte le tolleranze richieste, quindi il test di arresto è soddisfatto subito: l'intervallo $[0,1]$ non viene suddiviso ulteriormente e la function usa solo i nodi $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$.

L'errore reale resta però grande, perché questi nodi non descrivono la parte significativa dell'integranda, concentrata molto vicino a 0. L'esempio mostra quindi un limite pratico dello stimatore computazionale usato dalla formula adattiva.

↳ Per riprodurre i valori numerici riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es30.m` .